UNIVERSITY OF CAMBRIDGE
Department of Computer Science and Technology

May 15, 2026

Efficient Coinductives through State-Machine Corecursors

William Sørensen

Gonville & Caius College

Submitted in partial fulfilment of the requirements for the Computer Science Tripos, Part II

Declaration

I, William Sørensen of Gonville & Caius College, being a candidate for the course, hereby declare that this report and the work described in it are my own work, unaided except as may be specified below, and that the report does not contain material that has already been used to any substantial extent for a comparable purpose.
Signed: William Sørensen
Date: May 15, 2026

Proforma

Word count: 8429

Acknowledgements

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit.

Contents

1 Introduction ... 1
1.1 Use cases ... 1
1.1.1 Infinite datastructures ... 1
1.1.2 Compiler semantics ... 1
1.1.3 Safe webservers ... 1
2 Preparation ... 2
2.1 Dependent type theory ... 2
2.2 Lean ... 2
2.3 Polynomial functors ... 4
2.3.1 Common polynomials ... 4
2.4 Recursion and Inductives ... 5
2.5 Corecursion and coinductives ... 5
2.5.1 In OCaml ... 5
2.5.2 Streams ... 6
2.5.3 The M-type ... 6
2.5.4 State-machine encoding ... 7
2.6 Inductive predicate ... 7
2.7 Coinductive predicate ... 8
2.7.1 Bisimilarity ... 8
2.8 Methodology ... 9
2.9 Tools used ... 9
2.10 Requirement Analysis ... 9
2.10.1 State-machine encoding (Core) ... 9
2.10.2 Non-termination-monad (Core) ... 10
2.10.3 ABI Type (Extention) ... 10
2.10.4 Interaction Trees (Extention) ... 10
2.10.5 Futumorphic productivity (Extention) ... 10
3 Implementation ... 11
3.1 Common ... 11
3.1.1 Transiteration ... 11
3.1.2 Expanding the progressive approximation theory ... 12
3.1.3 Polynomial equivalents ... 13
3.1.4 Natural isomorphisms of MvFunctors ... 13
3.2 Stream implementation ... 13
3.2.1 SME ... 13
3.2.2 PA ... 14
3.2.3 Proving the equivalence ... 14
3.3 State-machine encoding of M-Types ... 15
3.4 Rephrasing bisimilarity for M-types ... 15
3.4.1 PreM ... 16
3.5 SM ... 16
3.5.1 Proving the equivalence ... 17
3.6 The ABI Type ... 17
3.6.1 Weak univalence ... 18
3.6.2 Relation to Shrink and further universe transforming types ... 18
3.6.3 HpLuM ... 18

3.7 NTMonad ... 19
3.8 Interaction trees ... 19
3.8.1 The ITree Monad ... 20
3.8.2 Weak bisimilarity ... 21
3.8.3 A formally verified optimization ... 21
3.9 Futumorphic Productivity ... 22
4 Evaluation ... 24
4.1 State-machine encoding ... 24
4.1.1 Performance between SME and PA ... 25
4.2 ABI Type ... 26
4.3 Interaction trees ... 26
4.4 Non Termination Monad ... 26
4.4.1 ITrees as the NTMonad ... 26
4.5 Futumorphic productivity ... 26
4.5.1 Case studies ... 26
5 Conclusions ... 27
Bibliography ... 28
6 Appendicies ... 30
6.A Project import graph ... 32
7 Proposal ... 34

List of figures

Figure 1 Inference rules for Π and Σ types . . . 2
Figure 2 Curry-Howard correspondance . . . 2
Listing 1 Definition for `Acc` taken from Lean's GitHub [1] . . . 3
Figure 3 Gcd as Source, Kernel and LCNF . . . 3
Figure 4 Formal definition of polynomial functor . . . 4
Figure 5 Projection polynomial . . . 4
Figure 6 Constant polynomial . . . 4
Figure 7 Product polynomial . . . 4
Figure 8 Sum polynomial . . . 5
Listing 2 Definition of list in OCaml . . . 5
Listing 3 Definition of colist in OCaml . . . 6
Figure 9 Agreement of two trees of height 1 and 2 . . . 6
Listing 4 PA Stream in Lean . . . 7
Figure 10 Evenness inductive predicate, from first principles . . . 8
Figure 11 Two bisimilar state-machines . . . 8
Figure 12 . . . 11
Listing 5 Transliteration between universes of types . . . 12
Listing 6 Natural isomorphisms on MvFunctors . . . 13
Figure 13 Key code involved in Stream equivlance . . . 15
Figure 14 Equivalent bisimilarity definitions . . . 15
Figure 15 Universe transforming types. . . . 17
Figure 16 Weak univalence up to shrink . . . 18
Figure 17 Defintions of ITrees . . . 19
Table 1 Functions implemented in (L)ean and (R)ocq . . . 20
Table 2 Equations implemented in (L)ean and (R)ocq[1] . . . 20
Figure 18 Weak-bisimilarity for ITrees . . . 21
Listing 7 `Imp` and `simpl` . . . 22
Figure 19 Relationship between `Free` monad and `DT` . . . 23
Table 3 Functions implemented for the Free monad . . . 23
Table 4 Requirements and completion . . . 24
Figure 20 Performance of SME, PA, and Big representation . . . 25
Figure 21 Import graph of the lean component of the project as of 2026-04-04 . . . 33

[1]The annotations in the table are as follows:

* = strong bisimilarity rathe rthan equality in Rocq

† = proven using coinduction-up-to in Rocq

Chapter 1: Introduction

1.1 Use cases

1.1.1 Infinite datastructures

Often, a problem in programming requires leaving the finite. In cases we want to maintain safety properties of the language. For example:

- We cannot use termination as a criteria for infinite structures, so how do we maintain the totality of the language.
- One would also want functions written for a language like this to have the expected runtime; we want a constant time to unfold.
- We want an interface that lets us unfold and reason about definition just how we would any other function.

A concrete example of a use case for this would be reading from a UNIX file. Since we can for example have a program `cat` which echos out all it is fed, then what would the output of `yes | cat` be? This would have to be a stream.

1.1.2 Compiler semantics

Recalling L1[2], we gave it a big-step operational semantics. We later tried to implement an interpreter for this semantics to execute L1 programs. This required us to prove quite a few substantial coherences. An alternative way we could have tried would be to write down an interpreter, and simply use this as the semantics. Most proof assistants would not allow this though, as they require function totality. So we would want:

- A way to write an interpreter of a partial language, and be able to use this interpreter as a semantics in formal software.
- We would also want the asymptotics of the interpreter to be the same as the source language.
- We want to be able to encode effects in the language in addition to just non-termination.

1.1.3 Safe webservers

[2]TODO: Add reference to sem of prog

Chapter 2: Preparation

In this section I will discuss prerequisites for the implementation of the dissertation. I will discuss the background needed to understand the state-machine encoding, and other relevant topics.

TODO: Integrate these terms

Propositionally extensionality $\forall(PQ : \text{Prop}).(P \leftrightarrow Q) \to (P = Q)$
Univalence $\forall(PQ : \text{Type}).(P \simeq Q) \to (P = Q)$
Axiom K Every equality $p : a = a$, can only be reflexivity.

2.1 Dependent type theory

Recall back to discrete maths [2], how functions between sets $A, B : \textbf{Type}$, $A \to B$ correspond with a form of arbitrary product, $B^A \equiv \prod_{x:A} B$. We can generalize this by changing B to be a function $B : A \to \textbf{Type}$, then letting us write the Π-type[3] $\prod_{x:A} B(x)$. We can also do the dual construction, and get Σ-types. The inference rules for these can be seen in Figure 1. These rules should be familiar as they are the analogous to the rules for $\forall, \exists$ from the natural deduction system, their relationship can be seen in Figure 2. If you are unfamiliar with dependent types, I recommend viewing them using this relationship. For a more in depth look at dependent type theory, read [3].

$$\frac{A : \textbf{Type} \qquad B : A \to \textbf{Type}}{\prod_{x:A} B(x)}\,\Pi\text{f}$$
(a) Π-forming rule

$$\frac{\Gamma, x : A \vdash v : B(x)}{\Gamma \vdash \lambda x.v : \prod_{x:A} B(x)}\,\Pi\text{i}$$
(b) Π-intro rule

$$\frac{\Gamma \vdash f : \prod_{x:A} B(x) \qquad \Gamma \vdash v : A}{\Gamma \vdash fv : B[x \ / \ v]}\,\Pi\text{e}$$
(c) Π-elimination rule

$$\frac{A : \textbf{Type} \qquad B : A \to \textbf{Type}}{\sum_{x:A} B(x)}\,\Sigma\text{f}$$
(d) Σ-forming rule

$$\frac{\Gamma \vdash a : A \qquad \Gamma \vdash b : B[x \ / \ a]}{\Gamma \vdash \langle a, b, \rangle : \sum_{x:A} B(x)}\,\Sigma\text{i}$$
(e) Σ-intro rule

$$\frac{\Gamma \vdash v : \sum_{a:A} B(a) \qquad \Gamma \vdash a : A, b : B[x \ / \ b] \vdash x : C}{\Gamma \vdash \text{let } \langle a, b\rangle := v \text{ in } x : C}\,\Sigma\text{e}$$
(f) Σ-elimination rule[4]

Figure 1: Inference rules for Π and Σ types

$$\Pi, \Sigma \underset{\text{Categorification}^5}{\overset{\text{propositional truncation}}{\rightleftarrows}} \forall, \exists$$

Figure 2: Curry-Howard correspondance

2.2 Lean

Lean [1] is a popular dependently typed proof assistant, it is famous for a few reasons.
1. It has Mathlib [5], one of the world's largest repositories of formalized mathematics.
2. It has a rich tactic language in which most proofs are written.
3. It has a compiler, and can be used as a general purpose programming language.

We will mainly focus on point 3., as a consequence of this point, we sometimes have a tension between the logical and computation sides of a proofs/programs. A classic example of where this occurs is

[3]Some call this a 'dependent product', or 'dependent function', but this sometimes confuses people so I will simply call it either a Pi, or a forall.

[4]Technically this isn't the most general eliminator for Sigma's, as C can depend on the content of the type. For symmetry and simplicity I omit this.

[5]S. Awodey [4] calls this 'categorification', as it is only one direction of Curry-Howard. Figure 2 can also just simply be said as 'Curry-Howard', but I prefer to highlight the directions of the relationship.

in the elimination principle for strong-induction[6]. We talk about this being a difference between the kernel (the small safe piece of code we believe to be correct) and the compiler (which is much larger and more complex).

Lean requires all induction to be structural, so to do induction over a general well-founded relation, we need some inductive (Section 2.4), Lean calls this `Acc` and defines it as seen in Listing 1. On this they define a function `WellFounded.fixF` which allows for strong induction. This function is quadratic in its runtime, therefore as a consequence the kernel takes quadratic runtime to execute.

```
inductive Acc {α : Sort u} (r : α → α → Prop) : α → Prop where
  /--
  A value is accessible if for all `y` such that `r y x`, `y` is also accessible.
  Note that if there exists no `y` such that `r y x`, then `x` is accessible.
  Such an `x` is called a _base case_.
  -/
  | intro (x : α) (h : (y : α) → r y x → Acc r y) : Acc r x
```

Listing 1: Definition for `Acc` taken from Lean's GitHub [1]

It would be unacceptable for a program using strong recursion to be quadratic for a general purpose programming language, therefore the compiler removes the usage of `fixF`, and generates the expected calls in the intermediate representation (LCNF).

We can see that given the source-code Figure 3a, it generates a definition Figure 3b with `Nat.fix` (a variant of `fixF`), but in the output LCNF we get Figure 3c, which does not contain any mention of `Nat.fix`, but rather we see the recursive call, demonstrating how the logical and computation definitions diverge.

```
set_option trace.Compiler.result true in
@[semireducible]
def gcd (a b : Nat) : Nat :=
  if h' : b = 0 then a
  else
    have : a % b < b := Nat.mod_lt a
      (Nat.pos_of_ne_zero h')
    gcd b (a % b)
termination_by b
```

(a) GCD source code

```
/--
info: def gcd._unary : (_ : Nat) ×' Nat →
Nat :=
WellFounded.Nat.fix (fun x ⇒
PSigma.casesOn x fun a b ⇒ b) fun _x a ⇒
  PSigma.casesOn (motive := fun _x ⇒
    ((y : (_ : Nat) ×' Nat) → InvImage
(fun x1 x2 ⇒ x1 < x2) (fun x ⇒
PSigma.casesOn x fun a b ⇒ b) y _x → Nat)
→
      Nat)
    _x
    (fun a b a_1 ⇒
      if h' : b = 0 then a
      else
        have this := …;
        a_1 ⟨b, a % b⟩ …)
    a
-/
#guard_msgs in
#print gcd._unary
```

(b) generated kernel code

```
[Compiler.result] size: 9
  def gcd a b : tobj :=
    let _x.1 := 0;
    let _x.2 := Nat.decEq b _x.1;
    cases _x.2 : tobj
    | Bool.false ⇒
      let _x.3 := Nat.mod a b;
      dec a;
      let _x.4 := gcd b _x.3;
      return _x.4
    | Bool.true ⇒
      dec b;
      return a
```

(c) Generated intermediate representation

Figure 3: Gcd as Source, Kernel and LCNF

This general trick inspired the project, as it demonstrates that there is precedent in Lean4 to have separate logical and executional models. This is further highlighted by the existence of the `@[implemented_by]` attribute, which lets you off-load the runtime behaviour of a function to another possibly unsafe definition. When a definition is marked as `@[implemented_by]`, the codegenerator treats it as the other function when compiling the code. Safety of programs using `@[implemented_by]` is a very complex endeavour to verify, as Lean has a tactic called `native_decide` that runs the generated code outside of the kernel.

When talking about lean we also use the term *definitionally equal* to mean β,η-equivalent, and *propositionally equal* if two terms are identified by a gADT. Additionally we have *heterogeneous equality* where the types are only propositionally equal.

[6]This is now, for the case of natural numbers, fixed as of lean 4.27. https://www.youtube.com/watch?v=LOUbbiV0mWc https://lean-lang.org/doc/reference/latest/releases/v4.27.0/

2.3 Polynomial functors

A functor is a function `(α : Type) → F α` sending types to types, and a map between types to functions between the functor at those types `(α β : Type) → (α → β) → F α → F β`, such that the map distributes over composition and preserves identities.

Polynomial functors are functors with a collection of constructors (the head type), where the children correspond to how many of the polymorphic argument are wanted, alternatively they are functors generated by the grammar The formal definition can be seen in Figure 4.

$$P(\alpha) \triangleq (h : H) \times c_h \to \alpha$$

Figure 4: Formal definition of polynomial functor

A polynomial functor $P(\alpha)$ is given by: a head-type H, along with a child family c_h parameterised by the head $h : H$. A polynomial functor applied at type α, is a sigma-type of the head, and the child family [6].

This is a univariate polynomial functor, this dissertation will mainly focus on multivariate polynomial functors. Where rather than having a single child, we have a n-tuple of children for an n-polynomial functor.

Polynomial functors have all finite products (Figure 6, Figure 7) and coproducts (Figure 6, Figure 8). Polynomial functors have fixed points corresponding to `inductive`s, and cofixed-points as `coinductive`s [7], [8]. In the figures there are ellipsies, in reality multivariate polynomials operate on Type-vectors which come from the category formed by the products $\mathbf{Type}^n$ for some $n : \mathbb{N}$, these are a pain to work with as will be discussed in the implementation[7].

2.3.1 Common polynomials

Some examples of common polynomials are projection Figure 5, constant functors Figure 6, binary products Figure 7, and binary sums Figure 8. One crucial polynomial I am not providing code for which is complex to define here, is composition. Composition takes an n-polynomial, and n lots of m-polynomials, and outputs an m-polynomial. For example, taking the 3-polynomial F and 3 2-polynomial G_1, G_2, G_3, we get that the constructor of the composition takes a $F(G_1\alpha\beta)(G_2\alpha\beta)(G_3\alpha\beta)$. The polynomial notation for this can be found in [5], [8]. For those familiar, the composition polynomial works exactly like composition in primitive recursive functions,

```
inductive Prj (i : Fin n) (a_1 ... a_n : Type)
  where
  | mk (t : a[i])
```

(a) Inductive Notation

$$\lambda(i : \underline{n}).\langle 1, \lambda j \quad h.\ \text{if } i = j \text{ then } 1 \text{ else } 0\rangle$$

(b) Polynomial

Figure 5: Projection polynomial

```
inductive Const (T : Type) (a_1 ... a_n : Type)
  where
  | mk (t : T)
```

(a) Inductive Notation

$$\lambda(T : \mathbf{Type}).\langle T, \lambda i \quad h.0\rangle$$

(b) Polynomial

Figure 6: Constant polynomial

```
inductive Prod (A B : Type) where
  | mk : A → B → Sum A B
```

(a) Inductive Notation

$$\langle 1, \lambda i \quad ().1\rangle$$

(b) Polynomial

Figure 7: Product polynomial

[7]TODO: Add exact section when it is written

```
inductive Sum (A B : Type) where
  | inl : A → Sum A B
  | inr : B → Sum A B
```

(a) Inductive Notation

$$\langle \mathbb{B}, \lambda \begin{pmatrix} 0 & \text{true} & \mapsto & 1 \\ 1 & \text{false} & \mapsto & 1 \\ _ & _ & \mapsto & 0 \end{pmatrix} \rangle$$

(b) Polynomial

Figure 8: Sum polynomial

2.4 Recursion and Inductives

Recall algebraic datatypes from your favourite language (OCaml, Haskell, Rust). These are structures freely generated from a set of constructors. The classic example is a list, given as two constructors, `val cons : 'a → 'a list → 'a list` and `val nil : 'a list`, and a way to consume lists `val fold : 'a → ('b → 'a → 'a) → 'b list → 'a`. We write this as seen in Listing 2. Inductive datatypes are well-founded trees of constructors[8]. Any pure function that terminates, must by definition return a finite tree of constructors. This means fold is guaranteed to terminate too. In Lean, all functions are compiled into a call to some recursor for a type, for lists this recursor is a dependent generalization of fold. All structurally recursive functions can be compiled into a call to fold. We also have two operations mk and dest. These form an equivalence (J. Lambek [9] Corollary 2.4), giving us the result that the cofix-point of F, written as μF, is equivalent to $F(\mu F)$. This means we can unfold μF.

```
type ('a, 'b) listF = Nil | Cons of 'a * 'b
type 'a ilist = Mk of (('a, 'a ilist) listF)

let mk x = Mk (x)
let dest = function | Mk v → v
let rec fold bh ih = function
    | Mk(Nil) → bh | Mk(Cons(h, t)) → ih h (fold bh ih t)
```

Listing 2: Definition of list in OCaml

2.5 Corecursion and coinductives

Coinductive datatypes are datatypes we allow to be infinitely deep. Where inductive datatypes have a `fold : (('a, 'b) f → 'b) → (('a, 'g) f as 'g) → 'b` (rec) to consume them, coinductives have an `unfold : ('b → ('a, 'b) f) → 'b → (('a, 'g) f as 'g)` (corec) to produce them. The intuition for this, is to view it as if a fix F yields an X, then cofix F yields a lazy X[9]. Another way to view coinductives are as a state-machine, here the corecursor takes center stage, we can say a coinductive datatype is a datatype generated by some corecursor state-machine.

2.5.1 In OCaml

We can think back to what we did in Foundations of Computer Science, we made lazy lists simply by thunking recursive occurrences, this thunking allows infinite structures to exist in finite memory. This happens because OCaml is CBV. If we were in Haskell (or any call by name / push-value language), the definition of icolist would be that of ilist. Then all we need to do is add a definition of `unfold`, `mk` and `dest`. All these can be seen in Listing 3. We have a coinductive version of Lambek's theorem now too.

[8] In OCaml, Haskell and Rust, datatypes do not correspond directly to inductives since we can have non-well-founded trees, an example is `let rec degen = Cons((), degen)`.

[9] That is fix $(1 + (A \times -))$ gives lists, then cofix $(1 + (A \times -))$ gives lazy lists.

```
type 'a icolist = CMk of (unit → ('a, 'a icolist) listF)

let mk x = CMk (fun _ → x)
let dest = function | CMk v → v ()
let rec unfold gen v = CMk (fun _ →
    match gen v  with
    | Nil → Nil
    | Cons(h, t) → Cons(h, unfold gen t))
```

Listing 3: Definition of colist in OCaml

The keen eyed will notice and call out that `unfold` is not structurally recursive, nor even decreasing at all. This is the key difference between inductives and coinductives; coinductives can be 'infinitely big'. Despite not being terminating, the definition is *productive* (think of this as coterminating). Termination is having a complete structure in finite time, productivity is growing in finite time.

2.5.2 Streams

Streams are similar to lists, the only difference is they have no 'nil' constructor. As one might expect there cant exist any inductive of this form[10]. Though this is a perfectly well-defined coinductive. Streams and colists are also common in general purpose programming, Java has Stream, Rust and Python have Iterator. This backs up our use cases put forward in the introduction. We will give concrete implementations of them for each of the encodings as follows.

2.5.3 The M-type

The M type is the name given to the cofixed-point of a polynomial functor[11] ; possibly infinitely deep trees in which each layer is an unfolding of F. This means the implementations must have both: a corecursor $\text{corec} : \{\alpha : \text{Type}\} \to (f : \alpha \to F\alpha) \to \alpha \to MF$, and a destructor $\text{dest} : \text{M } F \to F(MF)$. These are the functions mentioned in Section 2.5.1. Often we also want to ship a way to `mk` a new layer, but we can define this from the two operators above.

When it comes to encoding M-types in Lean, there are a few different approaches. We will focus on two encodings of M-type's, the state-machine encoding Section 2.5.4 and Progressive Approximation Section 2.5.3.1 (as seen in [5]).

2.5.3.1 Progressive approximation encoding

A simple way to encode M-types, are as functions emitting trees that must at every level agree. This corresponds to the limit over the natural number category as the dual of J. Adámek [10] (see section "Free algebra algorithm"). Agreement is given by them being the same up to the previous depth as seen in Figure 9. I like to think about agreement as the structure 'crystalizing', since it can only 'grow' from the ends.

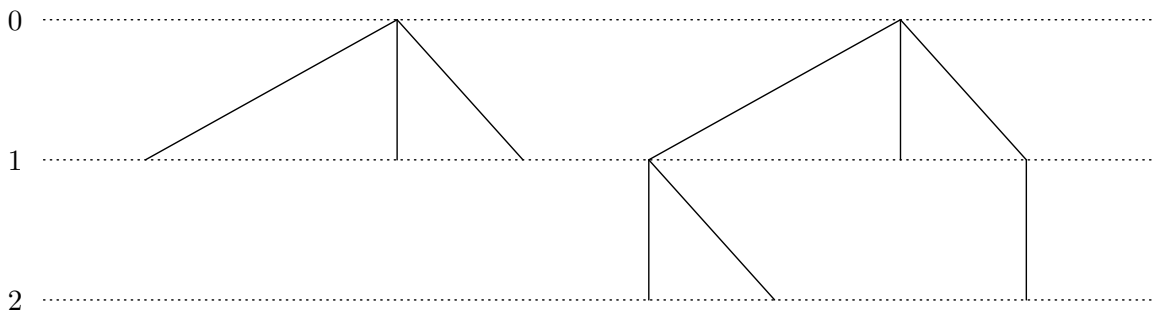


Figure 9: Agreement of two trees of height 1 and 2

[10]See the following lean

```
inductive SS (α : Type) where | cons : α → SS α → SS α
def SS.toEmpty : SS α → Empty | .cons _ tl ⇒ toEmpty tl
```

[11]Often phrased as types who's semantics are terminal F-coalgebra.

In Lean, this encoding is easy to set up. It can be seen in Listing 4. The main component which is hard to define is a coinduction theorem (Section 2.7.1). M. P. Fiore [11] is an insightful read on this topic. Luckily for us we do not need to worry about implementing and proving this as [5] already provides an implementation of this. This was ported to Lean4 as part of [7].

A major problem with this implementation is the complexity of access. As you can see in Listing 4 in the definition of `corec.o`, we have to regenerate the entire tree to get a corecursor to the next depth. There are a few different approaches to solve this that we will compare and contrast in Section 4.1.1.

```
def StreamF (α ρ : Type _) := α × ρ

inductive StreamF.Crystal {α : Type} {base : Type} : {n : Nat} → {m : Nat}
    → n.repeat (StreamF α) base → m.repeat (StreamF α) base → Prop where
  | base  : @Crystal _ _ 0 (m + 1) b v
  | consS : @Crystal _ _ n m t₁ t₂ →
      @Crystal _ _ (n + 1) (m + 1) ⟨h₁, t₁⟩ ⟨h₁, t₂⟩

structure PA (α : Type _) : Type 0 where
  obj : (n : Nat) → n.repeat (StreamF α) Unit
  cryst : ∀ n, StreamF.Crystal (obj n) (obj (n + 1))

namespace PA
def corec.o (gen : ρ → StreamF α ρ) (v : ρ) : (n : Nat) → n.repeat (StreamF α) Unit
  | 0 ⇒ .unit | n + 1 ⇒ (gen v).map id (o gen · n)
def corec (gen : ρ → StreamF α ρ) (init : ρ) : PA α where
  obj := corec.o gen init
  cryst n := by
    induction n generalizing init
    · exact .base
    case succ n ih ⇒
      unfold corec.o
      rcases gen init with ⟨a, b⟩
      change StreamF.Crystal (n := n+1) (m := n+2) ⟨a, corec.o gen b n⟩ ⟨a, corec.o gen b (n + 1)⟩
      exact StreamF.Crystal.consS (h₁ := a) (ih b)

def dest (x : PA α) : StreamF α (PA α) := ⟨
  (x.obj 1).1,
  (x.obj ·.succ ▷.snd),
  fun n ⇒ by
    have := x.cryst (n+1)
    dsimp
    generalize (x.obj (n + 1)) = a, (x.obj (n + 1 + 1)) = b at this;
    rcases a; rcases b; rcases this with (_|ih)
    exact ih
⟩
end PA
```

Listing 4: PA Stream in Lean

2.5.4 State-machine encoding

The state-machine encoding is the naïve way to implement the M-type. Given some polynomial functor F, the state-machine encoding is given by: some type $\alpha : \text{Type}$, a function $f : \alpha \to F\alpha$, and some witness $a : \alpha$. Once this is done there are 3 key problems: 1) The object constructed is only weakly terminal, 2) the object created has no coinduction principle, 3) the object resides in a higher universe. Let us focus on the first two problems (the 3rd problem is tackled in Section 3.6), both of these are solved by quotienting over bisimilarity. The reason this works has to do with how bisimilarity 'hides' the generating type, making the only operation you can do to access the data a destructure.

2.6 Inductive predicate

An inductive predicate is a least fixed point; a universal quantification over all relations of a given shape. An example of an inductive predicate is evenness. This is defined as a higher-order predicate called the `Shape` in this case, we use this to restrict the relations we are talking about. Then we take the universal over this shape and we get what is shown in Figure 10. It can be shown that this definition is equal to the one generated by the expected Lean code.

```
def IsEven.Shape (P : Nat → Prop) : Prop :=
  P 0 ∧ ∀ n, P n → P (n + 2)
def IsEven (n : Nat) : Prop :=
  ∀ P, IsEven.Shape P → P n
def IsEven.z : IsEven 0 := fun P hP ⇒ hP.left
def IsEven.ss n (h : IsEven n) : IsEven (n+2) :=
  fun P hP ⇒ hP.right n (h P hP)
```

(a)

$$\text{shape}_P \triangleq P0 \wedge \forall n, Pn \to P(n+2)$$
$$\text{Even } (n) \triangleq \forall P, \text{shape}_P \to Pn$$
$$e_0 : \text{Even } 0 \triangleq \lambda P \text{ hP}.\pi_1(\text{hP})$$

(b)

Figure 10: Evenness inductive predicate, from first principles

2.7 Coinductive predicate

Given what we saw in the last section and how we have seen the duality for coinductive and inductive data; all we must do is dualize this construction. So we change the universal to an existential, disjuncts to conjuncts, and flip the arrows (each of these swap left and right adjoints). This gives us a proof principal with three components. We now need to define an invariant, we need to prove the invariant satisfies the restriction, and that our value is contained in the invariant.

Proving coinductive predicates is very challenging as it requires coming up with the invariant. This is a kind of proof I had no experience in, and it took a while to pick up. An additional difficulty is, if you get the invariant wrong, you have to start completely again and try to come up with something better. This means that for some of my proofs I wrote hundreds of lines of code that have not made it into the dissertation.

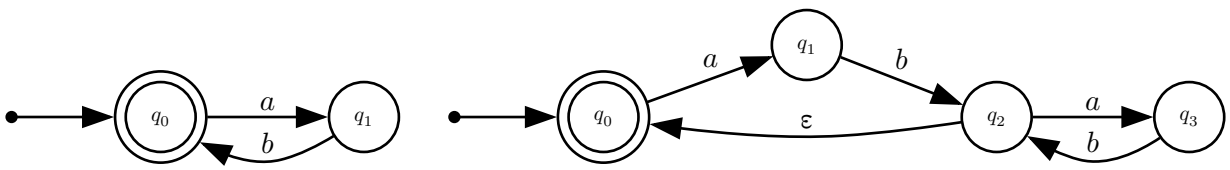
Some languages have facilities to make this better, these are collectively known as coinduction-up-to. These are techniques where you can reuse earlier proofs and automatically expand the invariant. For Rocq a popular library for this is Paco[12], a lot of harder coinduction proofs use this technique. Lean does not have this feature, and it is out of scope for the dissertation. In its place I spend a lot of time picking the shape of the Invariant I need.

2.7.1 Bisimilarity

Bisimilarity is an equivalence relation on coinductive types. Two coinductives are bisimilar if, for every possible choice in a structure A, there exists a corresponding choice in structure B, such that you can repeat this procedure.

Bisimilarity is not a well-founded relation. This means we need to define it as a coinductive predicate rather than an inductive one. Then the proof principal for this will be giving an invariant, showing the invariant is a bisimilarity, and our values are contained within the invariant.

A classic example would be to check if the two state-machines are equal. In Figure 11 you see two state-machines for two different regexs. One could use bisimilarity to show these are the same. The invariant for this would probably be $(a = q_0 \wedge b = q_0) \vee (a = q_0 \wedge b = q_2) \vee (a = q_1 \wedge b = q_1) \vee (a = q_1 \wedge b = q_3)$. This also contains the initial state $a = q_0 \wedge b = q_0$. We also need to show the invariant is a bisimilarity which is a bit more involved.



(a) DFA for `(ab)*`

(b) NFA for `(ab(ab*))*`

Figure 11: Two bisimilar state-machines

[12]TODO CITE

On streams bisimilarity corresponds to saying that the heads of the streams are equal and then the tails are bisimilar. A coinductive type has a coinduction principle if bisimilarity implies equality. Streams and in general cofixed-points of polynomials satisfy this.

2.8 Methodology

Writing in a proof assistant is quite different to writing in a general purpose language, this is especially the case in Lean as it is a hybrid of both styles. A notable and immediate difference is the prevalence of software testing of programs. Rather than having tests that can be flakey or incomplete, in Lean you have checked proofs.

In a conventional programming language, you can follow Test Driven Development where you constantly refine the specification with additional tests, which you check as you go through the code. In Proof Driven Development (PDD, [12]) you work similarly to TDD, but now when you have your function, rather than knowing it works for finite cases, you can be guaranteed that it satisfies the equation. Furthermore, the proofs can serve to enrich the main artifact. This will be seen in the ITree implementation as well as the futumorphic productivity. Lean makes working in this style a delightful experience using constructs such as `#check`, `#print` and `#eval`, one can see exactly what a function is and what it is doing. These can also be turned into conventional tests using `#guard_msgs in`, which raises a compile error upon an output changing. PDD was originally demonstrated for Idris, which in many ways is similar to Lean as it is a hybrid language, therefore I will apply this approach as I write the code.

Lean also has great automation features which I use at many points in the dissertation, these are the unification solver `exact?` and SMT solver `grind`. These both stand as principled (as opposed to LLM Slop) ways to develop programs and proofs that would otherwise be tedious for a human. I use these regularly to speed up the development process.

2.9 Tools used

To aid the development and communication process I used `git`, hosting the repository on GitHub. I also shipped a compiled version of this dissertation publicly, so my supervisors could read it and have it update live on the web. The dissertation is written in **typst** as it is a feature rich, well documented, and popular TeX alternative.

2.10 Requirement Analysis

To expand on the success criteria given in the proposal, I specified the project using MoSCoW. Here the success criteria have become MUSTs, and improvements have become SHOULD. I have done this for the core and each of the extensions of the project.

2.10.1 State-machine encoding (Core)

SM1 The SME Stream MUST be implemented in its high universe form.
SM2 The equivelence between SME Stream and PA Stream MUST be implemented.
SM3 The SME M MUST be implemented in its high universe form.
SM4 The SME M MUST be constant time under destructuring.
SM5 The equivelence between SME M and PA MUST be implemented.
SM6 The SME M MUST be able to express the NT monad, Section 2.10.2.

SS1 The SME M SHOULD have a coinduction principle.

SC1 The SME M COULD have a low universe shifted version Section 2.10.3.
SC2 The SME M COULD have an Interaction tree library Section 2.10.4.
SC3 The SME M COULD have a Productivity Transform Section 2.10.5.
SC4 The SME M COULD be zero cost.

SW1 The SME M WONT have a syntax macro as in [7].
SW2 There WONT be any work towards coinduction-up-to systems.

2.10.2 Non-termination-monad (Core)

NM1 The NTMonad MUST be implemented using the SME.
NM2 The NTMonad MUST have a monadic bind and return.

NS1 The NTMonad SHOULD be a lawful monad.

NC1 The NTMonad COULD be set up to use Section 2.10.5.

2.10.3 ABI Type (Extention)

AM1 The ABI Type MUST be implemented.
AM2 The ABI Type MUST have an eliminator.
AM3 The ABI Type MUST have the computational behaviour of the higher type.

AC1 The ABI Type COULD be zero-cost.

2.10.4 Interaction Trees (Extention)

IM1 ITrees MUST be implemented.
IM2 ITrees MUST have strong bisimilarity.
IM3 ITrees MUST have bind and return constructs.
IM4 ITrees MUST have KTrees (continuation trees).

IS1 ITrees SHOULD have flow combinators implemented.
IS2 ITrees SHOULD have a coinduction principle.
IS3 ITrees SHOULD be a lawful monad.

IC1 ITrees COULD have weak bisimilarity implemented.
IC2 ITrees COULD have the monadic interpretation.

IW1 ITrees WONT implement the family of effects put forward in [13].

2.10.5 Futumorphic productivity (Extention)

FM1 Futumorphic productivity MUST have an in-universe corecursor.
FM2 Futumorphic productivity MUST have an injector.

FS1 Futumorphic productivity SHOULD have proof-principles for unfolding the corecursor.
FS2 Futumorphic productivity SHOULD have proof-principles for the injector.

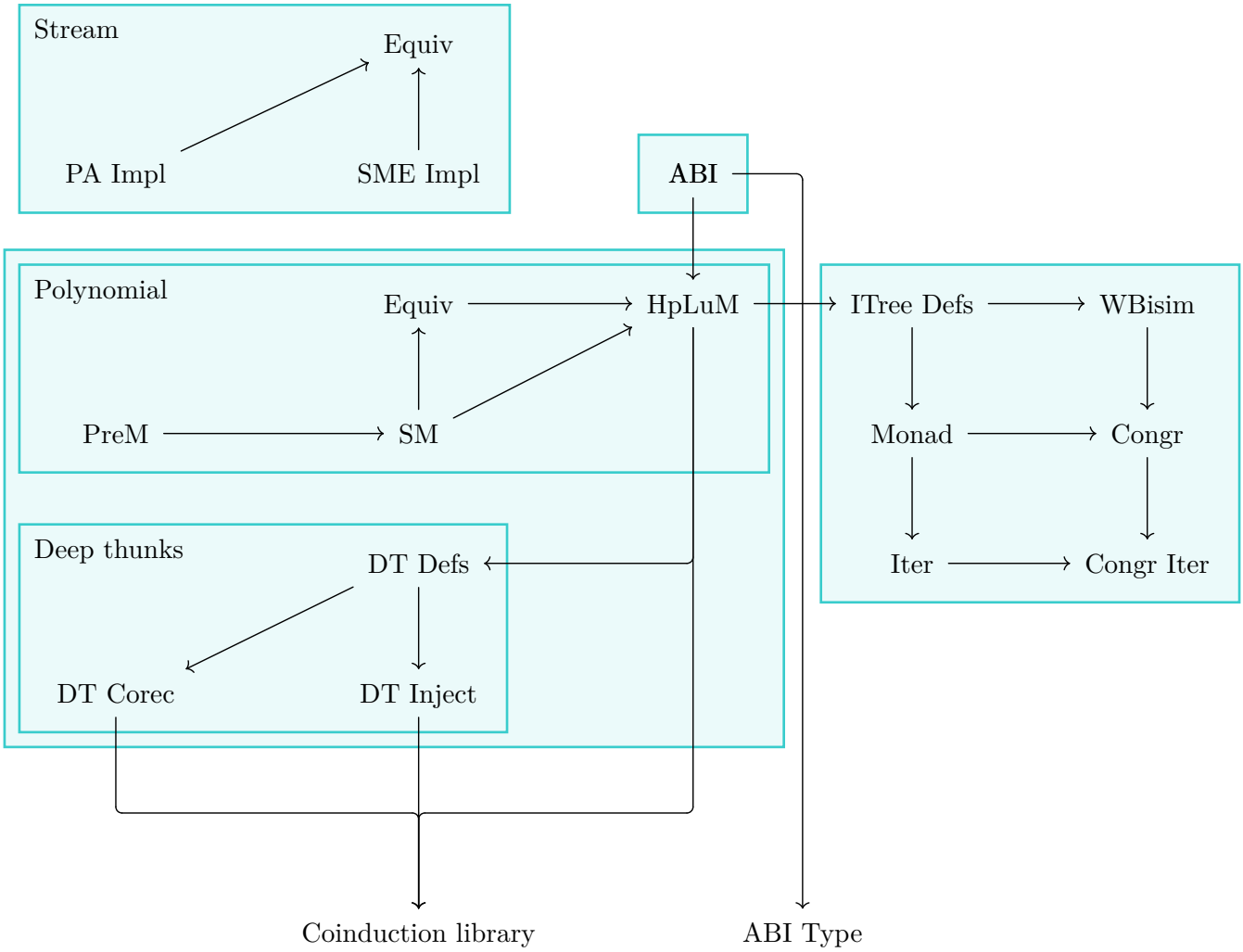
FC1 Futumorphic productivity COULD have a cross universe corecursor.
FC2 Futumorphic productivity COULD have a universe transliterator.
FC3 Futumorphic productivity COULD have the ability to reason about *closed* trees.

FW1 Futumorphic productivity WONT be as expressive as [14].

Chapter 3: Implementation

This chapter describes the implementation of each of the requirements stated in Section 2.10. I will break it down into a common section, then go by individual components. I will also mention which parts are in process of being upstreamed and which parts are already in Mathlib [5].

The Lean component of this repository, and how they relate to each of the requirements can be seen in the list below. Additionally the import graph can be seen in Section 6.A



3.1 Common

3.1.1 Transiteration

A family of functions, that keep solving problems throughout this dissertation are what I call transliterations. Given some parameter span X[13], either in some **Type**, or more commonly over some universe $\mathcal{U}$. We define a transliteration on X as a family of functions $t_{a,b} : Xa \to Xb$, such that they are closed under composition $t_{b,c} \circ t_{a,b} = t_{a,c}$[14], and the self-path is identificaion $t_{a,a} = \mathrm{id}$.

[13]This not necessarily a type, as lean does not have omega-types (Setω from Agda).

[14]This is very similar to saying it's an idempotent, but technically $t_{b,c}$ and $t_{a,b}$ arent the same function.

The trivial instantiation of a transliteration is a `cast`[15], here we pick X as equal types, obviously `cast aa = id` and `cast bc ∘ cast ab = cast (ab.trans bc)`. One could even say that a transliteration is a function that behaves like `cast`.

An example we will keep defining is universe transliteration, here we take $X = \text{ULift}_{(-)}$, using this we define the transiteration as seen in Listing 5. This is the closest we get to having omega-types and cumilativity in lean; as long as the function is applied at a usage of ULift it all just works. We will use this function to define transliteration between universe-lifted polynomials, we will see more of this in Section 3.1.2.1.

```
universe u v w
variable {A : Type u} {x : ULift A} {a : A}

@[inline] def transliterate : ULift.{v} A → ULift.{w} A := ULift.up ∘ ULift.down

@[simp] theorem transliterate_idempotent
    : transliterate ∘ transliterate = transliterate (A := A) := rfl
@[simp] theorem transliterate_noop
    : transliterate x = x                                    := rfl
```

Listing 5: Transliteration between universes of types

Another major usage of a transiteration was used when defining the eliminator for `ABI`, I will discuss this more when we get to Section 3.6. This was where I first discovered transliteration, and made it possible to define a universe-polymorphic eliminator.

3.1.2 Expanding the progressive approximation theory

Addressing: , Path: `sme/Sme/ForMathlib/{PFunctor/*,TypeVec.lean}`

During the feasability assesment I noticed that, in the current formalised theory of polynomials, the equivalence wouldn't even type-check. This stemmed from a problem with the corecursive principle for the M-type in the old implementation. corec : $\{\alpha : \text{TypeVec}.\{\mathcal{U}\}n\} \to \{\beta : \text{Type } \mathcal{U}\} \to (g : \beta \to P(\alpha ::: \beta)) \to \beta \to M\alpha$[16] Mathlib [5]. The problem here is that both α and β have to both reside in $\mathcal{U}$. Solving this is done through the next two sections.

3.1.2.1 Universe lifting of polynomial functors.

The main problem caused here comes from the fact that lean isnt cummulative. This means it is impossible to express a universe heterogeneous typevector. In other words $\alpha ::: \beta$ is only typable if $\alpha : \text{TypeVec}.\{\mathcal{U}\}n$ and $\beta : \text{Type } \mathcal{U}$. The natural way of solving this is using the supremum in universe levels you get from $\text{ULift} : \text{Type } \mathcal{U} \to \text{Type } (\max \mathcal{U}\mathcal{V})$. This means we can have $\beta : \text{Type } \mathcal{U}$ and $\alpha : \text{Type } \mathcal{V}$, then ulift both of them to a common universe $\text{ULift } \alpha ::: \text{ULift } \beta : \text{TypeVec}.\{\max \mathcal{U}\mathcal{V}\}(n + 1)$[17].

Noticable the next hurdle we encounter is that PFunctors are restricted to a universe level. Recall the definition from Section 2.3. Observe how for a $\text{MvPfunctor}.\{\mathcal{U}\}n$, we require that both the head and child reside in $\mathcal{U}$. This will also cause problems, as looking back at the definition of the corecursor, we will require P to be able to accept $\text{ULift } \alpha ::: \text{ULift } \beta$. If we do not add the ability to lift P, the unifier will force $\mathcal{U} = \mathcal{V}$, thereby invalidating all the work we did in the previous section. Luckily lifting a PFunctor is relatively easy. We define it as $\text{ULift } P \triangleq \langle \text{ULift } P.1, \lambda x \mapsto \text{ULift } (P.2x.\text{down}) \rangle$. This works and now we can move on to generalizing the corecursor. This defintion is also natural in the expected way.

[15] Or if you read [3], this is called `transport`.

[16] https://github.com/leanprover-community/mathlib4/blob/7a60b315c7441b56020c4948c4be7b54c222247b/Mathlib/Data/PFunctor/Multivariate/M.lean#L152-L154

[17] Note we overload ULift as a notation to refer to lifting TypeVecs as well

3.1.2.2 Generalizing the corecursor

Addressing: , Path: `[UPSTREAMED],sme/Sme/PFunctor/ForMathlib/M.lean`

Now with all the work in the previous section, first we generalize a helper function[18], then we can define corecU : $\{\alpha : \text{TypeVec}.\{\mathcal{U}\}n\} \to \{\beta : \textbf{Type } \mathcal{V}\} \to (g : \beta \to \text{ULift } P(\text{ULift } \alpha ::: \text{ULift } \beta)) \to \beta \to M.\{\mathcal{U}\}\alpha$. Notably we are able to fit the object into $\mathcal{U}$ (this will not be the case for the SME).

`corecU` and `dest` satisfy an unfolding equation. This is more complex than it used to be as we now need to lower before we continue with the mapping.

3.1.3 Polynomial equivalents

Addressing: , Path: `sme/Sme/PFunctor/EquivP.lean`

Often when people talk about polynomials, they actually mean structures that are equivalent to polynomials. This is for example how [7] refers to polynomials. I found it useful to work like this as well, for this I made a type-class `EquivP` heavily inspired by how it was done in [7] Mathlib [5]. I took the definition of curried type-functions (CTFs) from [7] and implemented coherences for these for the common polynomials. I used this to make it cleaner to work with ITrees when I reached that point. A key difference is that I implemented it using the output CTF as an outParam, this means that I can synthesize the CTF from a given polynomials allowing for an easier user interface. Type class synthesis works similar to a prolog engine and `outParam` corresponds to the prolog mode `?`.

3.1.4 Natural isomorphisms of MvFunctors

Addressing: , Path: `sme/Sme/PFunctor/NatIso.lean`

A problem I found when working with isomorphisms of Mv(P)Functors was that isomorphisms needed tons of coherences proven about them. I observed that the coherences I was proving corresponded to naturality. To then solve this I defined natural isomorphisms of multivariate functors. To define this was relatively simple and can be seen in Listing 6. Using what I learnt in category theory, it was easy enough to prove the symmetric direction of `nat'`.

```
structure NatIso {n} (P Q : outParam (TypeVec.{v} n → Type u))
    [MvFunctor P] [MvFunctor Q] where
  equiv : ∀ {α}, P α ≃ Q α
  nat' {α β} {f : α ⇒ β} (x : P α) : f <$$> equiv x = equiv (f <$$> x)
```

Listing 6: Natural isomorphisms on MvFunctors

3.2 Stream implementation

Addressing: SM1, SM2, Path: `sme/Sme/Stream/*`

As proving these equivalences is challenging I decided I would start by implementing it for the special case of streams. Streams are the text-book coinductive datatype that most people know as mentioned in Section 2.5. Therefore I expected this to be pedagogical to implement.

3.2.1 SME

Addressing: SM1, Path: `sme/Sme/Stream/SDefs.lean`

First I set up a class (can't be a **Type** as it varies through universe levels) of prestreams, these correspond to the corecursive principle holding their type.

[18] Done in PR #30400 to Mathlib [5]

I had to define hd and tl for streams here corresponding to the destructors of streams. Care had to be taken to ensure that the definition would work under a quotient as well.

Then I defined a bisimilarity relation; heads being equal and tails bisimilar as seen in Section 2.7.1. I proved this was an equivalence relation (reflexive, symmetric, transitive). Using this I defined a setoid on PreStreams.

The definition of an SME Stream is then preobjects quotiented by this setoid. Quotients are famously a pain to work with. When lifting the destructors from PreStreams to Streams I have to go through the lifting function of Quotients. When lifting to a quotient one has to provide the lifting function, then a proof that the function is stable under quotienting. Initially these proofs were in tactic-mode, but were rewritten in term-mode for readability. The corecursor was simpler to define, it is just the constructor for PreStreams under a Quotient introduction.

The next step was defining a coinduction principle, in the code this is called `bisim` to align to convention with Mathlib [5]. The proof of this proceeds by Quotient soundness anc can be found in `SDefs.lean`.

Parts of this definition can be seen in Figure 13b.

3.2.2 PA

Addressing: , Path: `sme/Sme/Stream/PDefs.lean`

To implement PA Streams, I had to first implement the Stream's base functor, this is quite easy for streams as they have one constructor (so a head of 1), and each of the families only hold one instance of the value (so child $\lambda i().1$). From here I defined the destructors of streams this is as simple as calling the child with the correct indicies.

For symetry I also defined a syntactically identical bisimilarity relation on PA Streams, and for this I also prove a coinduction principle for PStreams of this relation. This proof proceeded by using the coinduction principle on general polynomials, which is a very tideous principle to work with as it requires unfolding the polynomial. When it was done I cleaned it up and this can be seen in `PDefs.lean`.

The corecursor wasn't as simple as for the case of SME Streams either, though all it required was doing a series of pattern-matches to get the right structure. I will do something similar like this for ITrees and the NTMonad as well.

Parts of this definition can be seen in Figure 13a.

3.2.3 Proving the equivalence

Addressing: SM2, Path: `sme/Sme/Stream/Equiv.lean`

The functions of this equivalence correspond to: the corecursors parameterised by the destructors of the opposite type. Proving that these are inverses was slightly involved, I toyed around with a few different relations trying to make it work. In the end I landed on the quite straight forward equality as seen in the Figure 13c. This solved very nicely and made the entire proof quite small after cleaning.

Having done this I was ready to approach the case of the polynomial. This turned out to be harder than I expected for 2 reasons.

1. Observant readers may notice the statement I proved is subtily the wrong statement. At the time I did not see this, but the equivalence should not be `SStream.{u, u} A ≃ PStream A`, but rather `SStream.{u, max u v} A ≃ PStream A`. This statement is actually dramatically harder to prove.

2. When you look at the proofs, in the bisimilarity there are `rfl`s, this is because the statements are **definitionally** equal (β,η-equivalent). For general polynomials this will not be the case and also explode ever so slightly.

```
def PStream.Base : MvPFunctor 2 :=
  ⟨PUnit, fun _ _ ⇒ PUnit⟩

def PStream (A : Type u) :=
  MvPFunctor.M PStream.Base fun _ ⇒ A

namespace PStream

variable {A : Type u}

def hd (x : PStream A) : A :=
  x.dest.snd (.fs .fz) .unit

def tl (x : PStream A) : PStream A :=
  x.dest.snd .fz .unit

coinductive Bisim (A : Type _) : PStream A →
PStream A → Prop
  | step {a b : PStream A}
    (cont : Bisim a.tl b.tl)
    (h : a.hd = b.hd)
    : Bisim A a b
```

(a) PA Stream

```
def SStream (A : Type u) := Quotient
  (PreStream.setoid A)

namespace SStream

variable {A : Type u}

def hd : SStream A → A :=
  Quotient.lift
    PreStream.hd
    fun _ _ ⟨_, his, rab⟩ ⇒
      match his rab with | .step _ h ⇒ h

def tl : SStream A → SStream A :=
  Quotient.lift (Quotient.mk _ ∘ PreStream.tl)
    fun _ _ ⟨r, his, rab⟩ ⇒
      match his rab with
      | .step h _ ⇒ Quotient.sound ⟨r, his,
h⟩

coinductive Bisim (A : Type _) : SStream A →
SStream A → Prop
  | step {a b : SStream A}
    (cont : Bisim a.tl b.tl)
    (h : a.hd = b.hd)
    : Bisim A a b
```

(b) SME Stream

```
def Sme.Stream.equiv.{u} {A} : SStream.{u, u}
A ≃ PStream A where
  toFun :=  PStream.corec SStream.dest
  invFun := SStream.corec PStream.dest
  left_inv _x := SStream.bisim ⟨
    fun a b ⇒ a = .corec PStream.dest (.corec
SStream.dest b),
    fun {a _b} (hab : a = _) ⇒
      hab ▸ .step rfl rfl,
    rfl
  ⟩
  right_inv _x := PStream.bisim ⟨
    fun a b ⇒ a = .corec SStream.dest (.corec
PStream.dest b),
    fun {a _b} (hab : a = _) ⇒
      hab ▸ .step rfl rfl,
    rfl
```

(c) Equivalence

Figure 13: Key code involved in Stream equivlance

3.3 State-machine encoding of M-Types

Addressing: SM3, SM5, Path: `sme/Sme/M/*`

Recalling back to Section 2.5.4, we can seeing how we proceeded in Section 3.2.1, we know how to proceed. First make a class of preobjects, then define a bisimilarity relation over these objects, make the object by quotienting this relation.

3.4 Rephrasing bisimilarity for M-types

Addressing: SS1, Path:

Coming into this dissertation, I implemented almost all of it using the definition of bisimilarity given in Mathlib [5] as Figure 14a. This definition turned out to be very hard to work with. Later in my dissertation I found an equivalent formulation using the universal property of the terminal object, since bisimilarity requires all but the last family to be equal, I simply equalize the objects last families. This reduces the up-front cost of proving an M-type bisimilarity, as previously one had to instantiate `a f f₁ f₂`, finding these values was very difficult and time consuming as they usually required partially unfolding the definition. The definition given in Figure 14b ended up being much easier to use. One of the reasons for this is that all the proofs about mappings of polynomials can be applied in a simp to get the preliminatry proof. This makes a lot of bisimilarity statements easier to work with.

```
def MvPFunctor.M.bisim (R : P.M α → P.M α → Prop)
  (h :
    ∀ (x y : P.M α), R x y → ∃ a f f₁ f₂,
      M.dest P x = ⟨a, TypeVec.splitFun f f₁⟩ ∧
        M.dest P y = ⟨a, TypeVec.splitFun f f₂⟩ ∧
          ∀ (i : (P.B a).last), R (f₁ i) (f₂ i))
  (x y : P.M α) (r : R x y) : x = y := ...
```

(a) Mathlib [5] definition

```
def bisim_map (R : P.M α → P.M α → Prop)
  (x : R a b)
  (h : ∀ s t, R s t →
    ∃ (x : (TypeVec.id ::: Function.const _ PUnit.unit) <$$> s.dest
      = (TypeVec.id ::: Function.const _ PUnit.unit) <$$> t.dest),
      ∀ v, R (s.dest.snd .fz v) ◁ t.dest.snd .fz
        ◁ cast (congr (congr rfl (Sigma.ext_iff.mp x).1) rfl) v)
  : a = b := ...
```

(b) New definition

Figure 14: Equivalent bisimilarity definitions

3.4.1 PreM

Addressing: SM3, Path: `sme/Sme/M/PreM.lean`

To define PreM we do exactly as we did for Streams, making sure to use the signature of `corecU` over `corec`. For the case of streams, we had two destructors `hd` and `tl`, in this case we have only one `dest`. Because of universe-level hell we cant define a nice signature like `dest : PreM P α → P (α ::: PreM P α)`, as `PreM P α` resides in **Type** $\max \mathcal{U}(\mathcal{V}+1)$ for some $\mathcal{V}$, instead we get the signature `dest : PreM P α → ULift.{max u (v+1)} P (ULift α ::: PreM P α)`. The definition of this requires transliterating the output of calling the generating function, as the generating function produces a $\mathrm{ULift}_{\max \mathcal{U}\mathcal{V}} P$.

In addition to the two functions `corec` and `dest` we have defined for `PreM`, I also need `ULifting` of `PreM`s. This is a function taking a `PreM.{u, v} P α` to a `PreM.{u, max v w} P α`. This was defined in such a way to take as much care as possible to make it easy to work under quotients… when we get to that section we can see to what extent I succeeded at this.

With all this plumbing done we can now move on to define bisimilarity. It is quite hard to define bisimilarity of `PreM`s using the `coinductive` syntax I set up. Instead, I use the relational lifting of multivariate functors. Relational lifting lets me reason about the children of a polynomial functor when instantiated correctly. The relational family I want to use for this is bisimilarity for the first argument, and identity for all of the other children. This is why I gave the first-principles definition of a coinductive predicate in Section 2.7, as we need to use this explicit cofixpoint construction to build the relation.

Proving this is an equivalence relation is much harder than for the case of streams. The reason for this is the much more roundabound definition of bisimilarity. Luckily Mathlib [5] has a lemma for working with liftR and PFunctors. These proofs can be found in `PreM.lean`. Once we have these we have that `PreM`s are setoids, and can proceed to define the quotiented version.

3.5 SM

Addressing: SM3, Path: `[UPSTREAMED],sme/Sme/{M/SM.lean,HEq.lean}`

I expected implementing the SME M-types to go as smoothly as implementing SME streams. Defining the corecursor was as simple, every other definition was much harder.

The destructor of SMs will be given by quotient lifting just as for streams. The function is also very similar, take the preobject destructor and equalize deeper occurances as to not leak the type. The stability of this function is proven by soundness of quotients for the corecursive case, and lifting the equality from bisimilarity for the other cases.

Proving the coinduction principle was dramatically harder. The reason for this, as with most things in this dissertation, is heterogeneous equalities. Since hetrogenous equalities have very few congruences[19], often I need to solve them on a case-by-case basis. As a consequence I ended up writing a few quite crucial lemmas, these include `dcongr_heq` which I upstreamed. As a consequence, this proof has both forward and backward sections. To save our sanity, I decided to rewrite it using Section 3.4, after doing this the proof simply fell out.

The next proof I had to do was with regards to universe lifting of SME M-types, this involves lifting the state-machine state rather than any part of the polynomial. This is the universe lifting function we defined for PreM types. Universe lifting of this component is needed for proving the generalized equivalence we stated in Section 3.2.3.

[19] An example is `(List A = List B) = (A = B)` is independent of lean.

3.5.1 Proving the equivalence

Addressing: SM5, Path: `sme/Sme/M/Equiv.lean`

Proving the equivalence on polynomials is much more challenging than proving it on streams. We know at least from streams, each of the directions will be given by each of their corecursors. One might expect to follow the type signatures and mostly you do, but Lean couldn't help with this, as it had to do higher-order unification. When Lean finally type checked one could see the functions are obvious as seen in FIGURE. We now need to prove that the two functions are inverses. When I was proving these equalities I was not very familiar with bisimilarities, and tried tens of invariants. Finally I landed on simply forcing their equalities. Once this was found, I had to prove it was a bisimilarity. I picked the head of the first type and was lucky that they both definitionally was of the same type. This means the equality for the children is homogenous. The other direction followed analougesly.

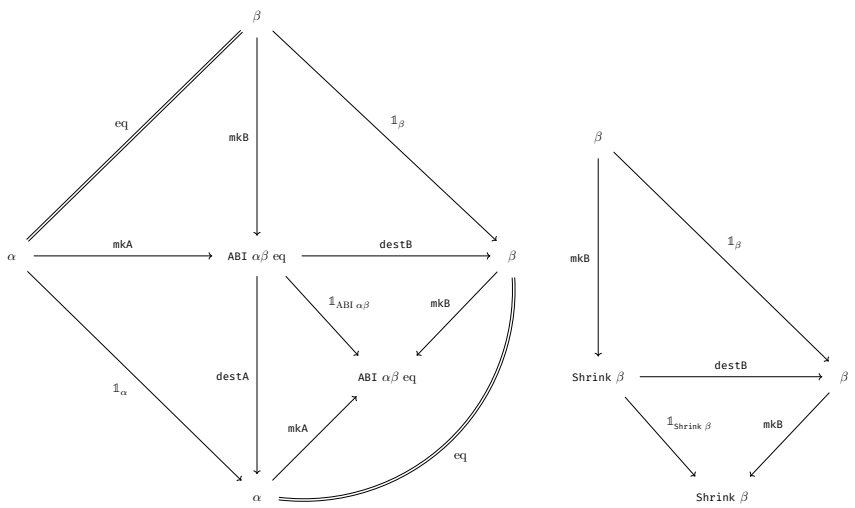
This was the main deliverable of the project, and would help make high performance low universe M -types (HpLuM)..

3.6 The ABI Type

Addressing: AM1, AM2, AM3, AC1, Path: `sme/Sme/ABI.lean`

Mathlib [5] has a type called `Shrink`. This is a type which takes an equivalence between a `A : Type u` and a `B : Type v`, and returns a `Type u`. It has one constructor `B → Shrink A B`, and one destructor `Shrink A B → B`, which are inerses. The definition given in Mathlib [5] is non-computable using choice, this makes it useless for us. Origionally I tried to make it computable by adding an `@[implemented_by]` to it, which turned out not to work. The reason this didnt work has to do with the fact that `Shrink` strangely satisfies the univalence axiom: given `Shrink A` and `Shrink B` along with `A ≃ B`, one can prove `Shrink A = Shrink B`. This follows from propositional extensionality as seen in FIGURE. Aaron Liu from the Mathlib [5] community used this in combination with quotients and trustCompiler to prove false. From this I understood that I would have to make my own variation of the type.

From this I decided to make my own variant I called the ABI type. Given an isomorphism $\mathrm{eq} : \alpha \cong \beta$ for some types α and β, I define the type `ABI` $\alpha\beta$ eq to have two constructor, eliminator pairs `mkA`, `mkB`, `destA` and `destB` satisfying the diagram seen in Figure 15a. Additionally I had an elimination principle satisfying expected β-rules with the two constructors. Through making the definition opaque I had quite a bit of problems with universe levels. I ended up solving this using a transliteration. This let me eliminate into any universe independent of $\mathcal{U}$ and $\mathcal{V}$.



(a) Operations on ABI (b) Operations on Shrink

Figure 15: Universe transforming types.

3.6.1 Weak univalence

$$\begin{tikzcd}
\beta \arrow[rr, equal, "\mathrm{eq}"] \arrow[dr, "\mathrm{mkB}"'] & & \alpha \arrow[dl, "\mathrm{mkA}"] \\
& \mathrm{ABI}\ \alpha\beta &
\end{tikzcd}$$

Figure 16: Weak univalence up to shrink

3.6.2 Relation to Shrink and further universe transforming types

TODO

3.6.3 HpLuM

Addressing: SC1, Path: `sme/Sme/M/HpLuM.lean`

After implementing the `ABI` type, it is time to put it to use. To do this, first I tried to simply use the equivalence, what I found was this leaked the universe of the carrier. I realised, I did not have to make a new equivlance for this, but rather paste the previous equivalence with itself. I called this equiv cross universe (`equivXU : SM.{u, max u v} P α ≃ SM.{u, max u w} P α`). One can observe this equivalence forms a transliteration. Additionally one might wonder, doesnt composing the equivalence with itself become a noop? To which the answer is no, as we are parameterised in universe levels, so technically these are different functions. This became a pain as the performance of this equivalence was $\mathcal{O}(n^2)$. To avoid this, I decided to mark this as `@[implemented_by, irreducible]`. Once this was done I could start implementing the High performance Low universe M.

The HpLuM is the ABI type, parameterised by the equivalence given in the last section at universes $\mathcal{U}$, $\mathcal{U}$. These universes prevent the carrier from being leaked. From here we define the corecursor of the same signature as in the `SM`, this uses a cocktail of universe changes, first one up using `SM.uLift`, then one down using `equivXU`, finally being passed into a `mkB` constructor of the `ABI` type. The next step is giving a `dest : HpLuM P α → P (α ::: HpLuM P α)`, this is done using the eliminator for the `ABI`-type. The reason I use this rather than just a `destB` (which is what `rec` compiles to), using the eliminator lets me prove the coherence of the SME and PA inline.

Once this was done I started proving equations of these methods. The first one was `dest_corec`, which has a nicer signature now. Followed by a coinductive principle. I also added an in-universe version of `corec` called `corec'`, this has the exact behaviour of the old non generalized corecursor on PA M types. In addition to this I added corecursor unrolling lemmas and mapping lemmas. From here I then proved co-Lambek's. Further that HpLuM actually is multifunctorial as the polynomial defintion, along with destructor lemmas for map.

Further I added functionallity to reason about polynomials equivalents, this is the reason I picked the CTF to be the outParam, as it lets these functions synthesize the correct CTF without annotations.

I also found I needed a defintion of ulifting of HpLuMs. Like everything to do with universe levels, this turned out to be a pain. I also proved naturality conditions of these.

The next thing I had to prove, was transportation of natural isomorphisms. This takes a natural isomorphisms between `P` and `P'`, and lifts it to a natural isomorphisms between `HpLuM P` and `HpLuM P'`. This will be used when we start working with futumorphic productivity.

When all of this was implemented, then I have a usable M-type library.

3.7 NTMonad

Addressing: NM1, Path: `sme/Sme/ITree/*`

The non-termination monad is a nice example of a coinductive datatype. It has two constructors `tau : NTMonad A → NTMonad A` and `val : A → NTMonad A`. The values of this type will always be either some value of `taus` then a `val`, or an a unique value `spin = tau spin`. One can think of these as `coroutines`, where they have a bind function that composes them.

I implemented them using by simply taking the cofixpoint of the Sum functor. This was mainly just to test that I understood what was needed to implement ITrees, once I understood ITrees I rather implemented them. This is adequet as `NTMonad A = ITree EmptyE A`, so it would be a waste to dobule up the implementation.

3.8 Interaction trees

Addressing: IM1, IM2, IM4, IS2, Path: `sme/Sme/ITree/*`

Interaction trees are the free-monad with non-termination as an effect [13]. In practice this means they can be used as a denotational object of imperative programs. They have 3 constructors, returning a value (monoidal unit), having a silent transition (for non-termination), and having a visible effect. This can be seen in Figure 17. The Lean defintion also includes a constructon of the polynomial equivalent. When KTrees are mentioned in the ITree paper [13], it is simply a function of the form `A → ITree E B`, for those familiar this is the object of the **k**leisli category.

```
CoInductive itree (E: Type -> Type) (R: Type): Type :=
| Ret (r: R)
| Tau (t: itree E R)
| Vis {A: Type} (e : E A) (k : A -> itree E R)
```

(a) Rocq defintion from [13]

```
@[grind]
inductive Base (ρ R : Type _)
  | tau (r : ρ)
  | ret (t : R)
  | vis {A : Type u} (e : E A) (k : A → ρ)

inductive PHBase : Type max v (u + 1)
  | tau
  | ret
  | vis (A : Type u) (e : E A)

def PBase : MvPFunctor.{max v (u + 1)} 2 where
  A := PHBase E
  B | .ret ⇒ !![PUnit, PEmpty]
    | .tau ⇒ !![PEmpty, PUnit]
    | .vis A _ ⇒ !![PEmpty, ULift.{max v (u + 1), u} A]
```

(b) Parts of the Lean defintion

Figure 17: Defintions of ITrees

ITrees have many operations and equations defined in the paper, These can be seen in Table 1, here I have annotatted all of the functions I have implemented from the paper. A notable difference is that in the Lean implementation strong bisimilarity implies equality, this makes using some of the proofs easier.

Interaction Tree Operations

- `ITree E A : Type` ^{LR}
- `tau : ITree E A → ITree E A` ^{LR}
- `ret : A → ITree E A` ^{LR}
- `vis {R} : E R → (R → ITree E A) → ITree E A` ^{LR}
- `bind : ITree E A → (A → ITree E B) → ITree E B` ^{LR}
- `trigger : E A → ITree E A` ^{LR}

Hetrogenous weak bisimilarity

- `eutt (r : A → B → Prop) : ITree E A → ITree E B → Prop` ^{R}

Strong and weak bisimilarity

- `_ ≅ _ : ITree E A → ITree E B → Prop` ^{LR}
- `_ ≈ _ : ITree E A → ITree E B → Prop` ^{LR}

Events and subevents

- `(e : E R)` ^{LR} — R is the result of event e
- `E +' F` ^{LR} — disjoint union of effects
- `class E -< F` ^{LR} — E is a subevent of F
- `trigger [E ≺ F] : E ⇝ ITree F` ^{R}

Parametric functions

`E ⇝ F := ∀ X, E X → F X` ^{LR}

Monadic interpretation

`interp [Monad M] [MonadIter M] : (E ⇝ M) → (ITreeE ⇝ M)` ^{LR}

Table 1: Functions implemented in (L)ean and (R)ocq

Structural Laws

- `tau t ≈ t` ^{LR}
- `t ≈ spin → t = spin` ^{L}
- `bind (tau t) k = tau (bind t k)` ^{LR*}
- `bind (vis e k₁) k₂ = vis e λy. bind (k₁ y) k₂` ^{LR*}

Equivalence relation

- `a ≈ a` ^{LR}
- `a ≈ b → b ≈ a` ^{LR}
- `a ≈ b → b ≈ c → a ≈ c` ^{LR}

Congruences

- `t₁ ≈ t₂ → tau t₁ ≈ tau t₂` ^{LR}
- `k₁ ≈ k₂ → vis e k₁ ≈ vis e k₂` ^{LR}
- `t₁ ≈ t₂ → k₁ ≈ k₂ → bind t₁ k₂ ≈ bind t₂ k₂` ^{LR}
- `k₁ ≈ k₂ → iter k₂ v ≈ iter k₂ v` $^{LR\,\dagger}$

Monad Laws

- `(ret v) >>= k = k v` ^{LR*}
- `t >>= ret = t` ^{LR*}
- `x >>= f >>= g = x >>= (f · >>= g)` ^{LR*}

Table 2: Equations implemented in (L)ean and (R)ocq[20]

3.8.1 The ITree Monad

Addressing: IM3, IS1, IS3, Path: `sme/Sme/ITree/Monad.lean`

ITrees form a monad, where the binding operation corresponds to whenever an itree returns, calling the continuation. The return constructor is the monads unit. From this to ensure it is lawful there is a set of proof obligations, these can be seen in Table 2.

[20] The annotations in the table are as follows:

* = strong bisimilarity rathe rthan equality in Rocq

† = proven using coinduction-up-to in Rocq

3.8.2 Weak bisimilarity

Addressing: IC1, Path: `sme/Sme/ITree/WBisim/*.lean`

Bisimilarity is often not sufficent for proving program equivalences. This comes from the fact that sometimes we care about more than just equality. For example the program `println 10` and `println (10 + 0)` are *equivalent*, but not syntactically equal. We can note that the effects produced by these programs are the same, one simply takes more steps to get there. This is where we rather consider the weak bisimilarity. One can think of weak bisimilarity is an equivalence relation modulo silent steps; two objects are observably the same effects but with a different tau count. The rocq defintion[21] and my defintion differs quite a bit. This is mainly because I decided to implement only hetrogenous weak bisimilarity, and also that in rocq they have coinduction-up-to making proofs much easier, in my defintion I need to 'bake in' as many principles as possible. This is the reason for the `refl` constructor. In the defintion I use `EStep` to mean some amount of taus, and `Step` to mean a productive step to some non-tau value. `EStep` forms a linear join-semilattice, `Step` inherits some of this structure, but is slightly easier to work with. One of the key pieces of structure helping this be closer to coinduction-up-to is the ability to skip taus in a proof, this is done using the principle `skip : EStep a c → EStep b d → Invariant E A R c d → Invariant E A R a b`, which operates directly on the invariant for the bisimilarity. This solves problems similarity to how it is done in the paper, it gives a principle where we can simply equations a lot by shifting them along the taus.

```
CoInductive euttF : itree E A → itree E B → Prop :=
| EqRet a b (REL: r a b) : euttF (Ret a) (Ret b)
| EqVis {R} (e : E R) k1 k2 (REL: ∀v, euttF (k1 v) (k2 v)) : euttF
(Vis e k1) (Vis e k2)
| EqTau t1 t2 (REL: euttF t1 t2) : euttF (Tau t1) (Tau t2)
| EqTauL t1 ot2 (REL: euttF t1 ot2) : euttF (Tau t1) ot2
| EqTauR ot1 t2 (REL: euttF ot1 t2) : euttF ot1 (Tau t2).
```

(a) Rocq definition

```
coinductive WBisim'
    (E : Type u → Type u) (A : Type v)
    : Base E (ITree E A) A → Base E (ITree E A) A → Prop
    where
  -- This case only exists for refl spin spin,

  -- in fact it could be this,
  -- but that would be harder to prove things about
  | refl {a b c} : EStep a c → EStep b c → WBisim' E A a b
  | ret {a b v} : Step a (.ret v)
      → Step b (.ret v) → WBisim' E A a b
  | vis {V e a b a' b'}
```

(b) Lean definition

Figure 18: Weak-bisimilarity for ITrees

In the ITree paper [13] it is proven weak-bisimilarity is an equivalence relation, a table of what is proven in each implementation can be found in Table 2. Out of these the ones marked with † have been proven using coinduction-up-to. These means they were particularly technical to prove and required defining helper predicates. The main instance of this is `iter`-congruence, for this I had to define two predicates `IterTrace` and `IterCotrace` for productive and spinning `iter`s respectively. This is used to define that if one of the generators of the iteration is weakly bisimilarity to the other generator, then if one is productive, the other is productive. Further it can be used to show that when they are productive, the values are related in the expected way. This proof was highly indirect and required the creative step of coming up with the (co)trace abstraction.

3.8.3 A formally verified optimization

Addressing: IC1, Path: `sme/Sme/ITree/WBisim/*.lean`

One of the main use cases of Interaction Trees is program verification. To demonstrate this I implemented a simple imperative language IMP as seen in Listing 7a, I give it the denotational semantics as Listing 7b, then I define an optimization `simpl` Listing 7c on this. All `simpl` does is simplifies constant expressions, the proof of this respecting the semantics is provided in the code, and is an example of how weak-bisimilarity can help prove an optimization correct.

[21] The syntax is slightly simplified for reading purposes here

```
inductive Imp (N : Type) : Type
  -- arith
  | add (a b : Imp N)
  | mul (a b : Imp N)
  | constN (n : Int)
  -- variables
  | var    (v : N)
  | assign (k : N) (a : Imp N)
  -- IO
  | print (a : Imp N)
  | seq (a b : Imp N)
  -- control
  | ite (c t f : Imp N)
  | while (c b : Imp N)
deriving DecidableEq, Repr
```

(a) Imp syntax

```
def sem {N} : Imp N → ITree (IOE ⊕, MapE N Int)
Int
  -- arith
  | .add a b ⇒ do
    let a ← sem a
    let b ← sem b
    return a + b
  | .mul a b ⇒ do
    let a ← sem a
    let b ← sem b
    return a * b
  | .constN n ⇒ .ret n
  -- var
  | .var v ⇒ .trigger ◁ .inr ◁ .getD v 0
  | .assign k v ⇒ do
    let v ← sem v
    let _ ← .trigger ◁ .inr ◁ .insert k v
    return 0
  -- effects
  | .print v ⇒ do
    let v ← sem v
    let _ ← .trigger ◁ .inl ◁ .print v
    return 0
  | .seq a b ⇒ do
    let _ ← sem a
    sem b
  -- control
  | .while c b ⇒ .iter (fun _ ⇒ do
      let c ← sem c
      if c ≠ 0 then
        let _ ← sem b
        return .inl .unit
      else
        return .inr 0
    ) Unit.unit
  | .ite c t f ⇒ do
    let c ← sem c
    if c ≠ 0 then sem t
    else          sem f
```

(b) Imp semantics

```
def simpl {N} (inp : Imp N) : Imp N :=
  flat ◁ pass inp
where
  flat : Int ⊕ _ → _
    | .inl v ⇒ .constN v
    | .inr v ⇒ v
  pass : _ → Int ⊕ _
    | .add a b ⇒
      match pass a, pass b with
      | .inl a, .inl b ⇒ .inl (a + b)
      | .inr a, b ⇒ .inr ◁ .add a (flat b)
      | .inl a, .inr b ⇒
        .inr ◁ .add (.constN a) b
    | .mul a b ⇒
      match pass a, pass b with
      | .inl a, .inl b ⇒ .inl (a * b)
      | .inr a, b ⇒ .inr ◁ .mul a (flat b)
      | .inl a, .inr b ⇒
        .inr ◁ .mul (.constN a) b
    | .constN n ⇒ .inl n
    | .print v ⇒
      .inr ◁ .print ◁ flat (pass v)
    | .var n ⇒ .inr ◁ .var n
    | .assign n v ⇒
      .inr ◁ .assign n ◁ flat (pass v)
    | .seq a b ⇒
      match pass a, pass b with
      | .inl _, b ⇒ b
      | .inr a, .inl b ⇒
        .inr ◁ .seq a (.constN b)
      | .inr a, .inr b ⇒ .inr ◁ .seq a b
    | .ite c t f ⇒
      match pass c, pass t, pass f with
      | .inl 0, _, f ⇒ f
      | .inl (.ofNat (_ + 1)), t, _
      | .inl (.negSucc _), t, _ ⇒ t
      | .inr c, t, f ⇒
        .inr ◁ .ite c (flat t) (flat f)
    | .while c b ⇒
      match pass c, pass b with
      | .inl 0, _ ⇒ .inl 0
      | .inl (.ofNat (_ + 1)), b
      | .inl (.negSucc _), b ⇒
        .inr ◁ .while (.constN 1) ◁ flat b
      | .inr c, b ⇒
        .inr ◁ .while c ◁ flat b
```

(c) simpl

Listing 7: Imp and simpl

3.9 Futumorphic Productivity

As mentioned in Section 2.5, every corecursive function has to be productive. In the current library defintion, this means we need to define the function using the corecursor `corec {β α} : (β → P (α ::: β)) → β → M P α`. This can be quite cumbersome as one can only define one layer of the coinductive datatype at one. During my internship under Alex Keizer and Tobias Grosser, I had an Idea of how to solve this, unfortunately then I didnt have time to implement this fully. The idea came from thinking about what choices one had when creating layers. After the first layer (which is required for productivity), we have a choice to either `recall` the corecursor from that point, or `continue` a deeper structure. We also observe that the argument we have to provide to `recall` would be the carrier. Putting these ideas together I created the structure Alex Keizer named 'DeepThunk' (`DT` for short). This structure, `DT P (α ::: β)`, was given as a cofix point of `μ ρ. (P α (Sum β ρ))` (given in `sme/Sme/M/DT/Defs.lean`). With difficulty, I then made a universe polynomial corecursor I called `dtcorec {β} : (β → DT P (α ::: β)) → β → M P α`, an injector `inject β : M P α → DT P (α ::: β)`, and a flattener `flatten : DT P (α ::: M P α) → M P α` These were painful to prove results about, but in the end I successfully proved an unfolding lemma about the corecursor, where it corrresponds to doing a mapping operation then flattening.

Seeing how much potential DeepThunks had, I asked Alex Keizer whether he had seen a structure like this before, and he pointed me to Z. Yang and N. Wu [15]. In this paper discusses different kinds of morphisms to construct datatypes. In the table I found futumorphisms, which looked exactly like

the structure I was looking for. These use the free monad and make a morphism `futu {β} : (β → P (α ::: Free P α β)) → M P α` [15], [16], [17]. The relationship betweem the `Free` monad and `DT` can be seen in Figure 19[22]. This demonstrates that `P (α ::: Free α β)` is exactly `DT`. A crucial difference is that `Free` has a nice, well defined notion of constructor and eliminator, while `DT` has nothing equivalent. This means as I wrote around 1700 lines for `DT`, almost all this functionallity fit into 700 lines for the futumorphism.

TODO Show layers

$$\underbrace{(P\alpha -) \lhd \overbrace{(\beta \oplus -) \lhd (P\alpha -) \lhd (\beta \oplus -)}^{\text{Free monad}} \lhd \ldots}_{\text{DeepThunk}}$$

Figure 19: Relationship between `Free` monad and `DT`

For the `Free` monad,

Free Monad Operations

- `Free P α β : Type`
- `recall : β → Free P α β`
- `cont' : P (α ::: Free P α β) → Free P α β*`
- `dest' : Free P α β → β ⊕ P (α ::: Free P α β)*`
- `corec' : (γ → β ⊕ P (α ::: γ)) → γ → Free P α β*`
- `map : Free P α β → (β → γ) → Free P α γ`
- `bind : Free P α β → (β → Free P α γ) → Free P α γ`
- `inject β : HpLuM P α → Free P α β`
- `flatten : Free P α (HpLuM P α) → HpLuM P α`
- `futu' : (β → P (α ::: Free P α β)) → β → HpLuM P α*`

Monad Laws

- `(recall v) >>= k = k v`
- `t >>= recall = t`
- `x >>= f >>= g`
  `= x >>= (f · >>= g)`

Functor Laws

- `id <$> x = x`
- `(f ∘ g) <$> x`
  `= f <$> g <$> g`

Table 3: Functions implemented for the Free monad

[22]Here ◁ is application as seen in OCaml.

Chapter 4: Evaluation

This chapter will discuss and evaluate each of the components of the dissertation, it will be split into distinct sections for each of the sections of the project. As mentioned in Section 2.8, I have been writing proofs verifying the correctness of my different components. I will refer to these throughout this chapter. Type signatures often are unable to encode performance (other than maybe something substructural), therefore for performance evaluations I simply write benchmarks as in any other programming language. There will be a comparison of ITree implementations. Finally an assessment of the ergonomics of futumorphic and deep-thunk productivy.

An overview of all the requirements laid out can be viewed Table 4.

Req.	Met	Evidence	Path
SM1	Y	Section 3.2	`Stream/SDefs.lean`
SM2	Y	Section 3.2.3	`Stream/Equiv.lean`
SM3	Y	Section 3.5	`M/Defs.lean`
SM4	Y	Section 4.1.1	-
SM5	Y	Section 3.5.1	`M/Equiv.lean`
SM6	Y	Section 3.7	`NTMonad/Defs.lean`
SS1	Y	Section 3.6.3	`M/{HpLuM,SM}.lean`
SC1	Y	Section 3.6.3	`M/HpLuM.lean`
SC2	Y	Section 3.8	`ITree/Defs.lean`
SC3	Y	Section 3.9	`M/{DT/*,Futu.lean}`
SC4	Y	Section 4.1.1	-

(b) State-machine encoding

Req.	Met	Evidence	Path
NM1	Y	Section 3.3	`NTMonad/Defs.lean`
NM2	Y*	Section 4.4.1	`ITree/Monad.lean`
NS1	Y*	Section 4.4.1	`ITree/Monad.lean`
NC1	Y	Section 4.5.1	`NTMonad/Defs.lean`

(a) NT Monad

Req.	Met	Evidence	Path
AM1	Y	Section 3.3	`ABI.lean`
AM2	Y	Section 3.3	`ABI.lean`
AM3	Y	Section 3.3	`ABI.lean`
AC1	Y	Section 4.2	`ABI.lean`

(c) ABI Type

Req.	Met	Evidence	Path
IM1	Y	Section 3.8	`ITree/Defs.lean`
IM2	Y	Section 3.8	`ITree/Defs.lean`
IM3	Y	Section 3.8.1	`ITree/Defs.lean`
IM4	Y	Section 3.8	`ITree/Defs.lean`
IS1	Y	Section 3.8	`ITree/Defs.lean`
IS2	Y	Section 3.8	`ITree/Defs.lean`
IS3	Y	Section 3.8	`ITree/Defs.lean`
IC1	Y	Section 3.8.2	`ITree/Defs.lean`
IC2	Y	Section 3.8	`ITree/Defs.lean`

(d) ITrees

Table 4: Requirements and completion

4.1 State-machine encoding

For the state-machine encoding, there are a few aspects to evaluate. For example performance, expressivity, and safety when compared to other implementations. The implementations I will test are the PA encoding from Mathlib [5], a mathematically optimal encoding, and an implementation by M. Sammler [18] which was started during this project. Notably this final implementation is built

on domain theoretic methods, meaning that rather than requiring productivity, all it requires is monatonicity. I will speak more about this when evaluating the futumorphic productivity.

4.1.1 Performance between SME and PA

After building the equivalence, and instantiating shrink, we now have the ability to compare the performance between 3 representations: The SME encoding (high performance, hpRuns), the PA encoding (slow, slRuns), and a hand made big implementation (bigRuns). The big implementation here serves as a theoretical optimum, with no overheads of universe alterations and so on. The big implementation is the underlying structure of the SM encoding, but without quotienting and all that is needed to make that nice to use. For SC4 to bd the case the SME would have to be within 1σ of the big encoding.

I implemented streams for the 3 different encodings, then I measured, using a monotonic clock, how long it took to m-samples destructure n-layers of the stream of natural numbers. For the PA I took $m = 10$ and swept $n \in [0, 200]$, and for SM and big encodings $m = 3$ and $n \in [0, 5000]$. I fit polynomials for each of these implementations, then I plot the average of the m samples for each layer n along with the fit. This generates the plot Figure 20. As we can see there is a discrepancy between the big and SM encodings. For an unknown reason the variance along each of these graphs pulsates, this effect is still relatively minor.

Reviewing the output graph we can see that the SME is $\mathcal{O}(1)$ under destructuring, as opposed to the PA encoding which is $\mathcal{O}(n)$. This is in line with the expected theoretical reasoning.

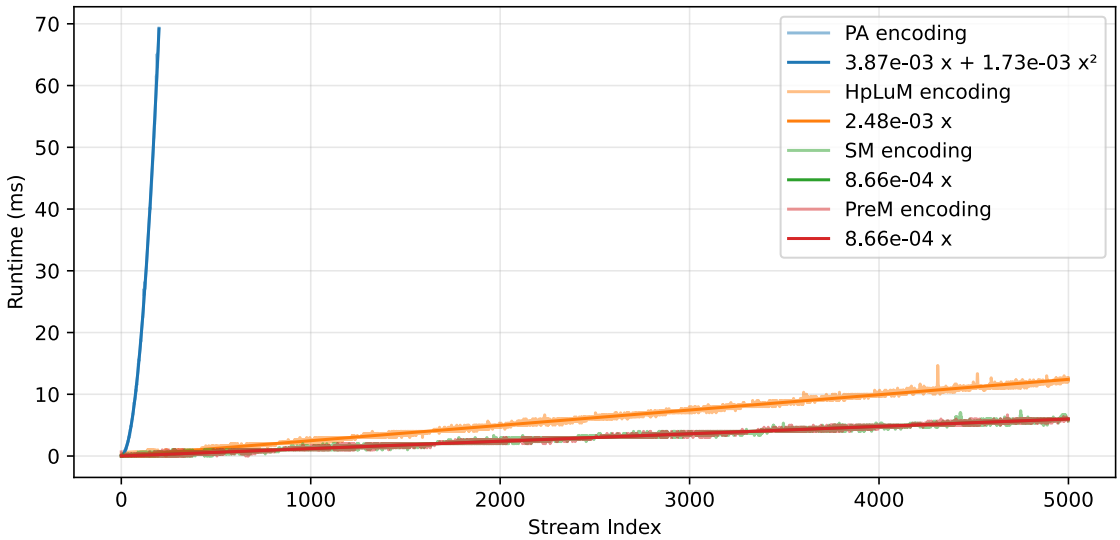


Figure 20: Performance of SME, PA, and Big representation

On the other hand, we can see that the big encoding is still around 4x faster. The issue causing this has to do with the destructor function needing to do universe lifting. This adds a fixed cost at each iteration, this is compared to the destructor for the big representation, which practically has to do nothing.

4.2 ABI Type

4.3 Interaction trees

4.4 Non Termination Monad

4.4.1 ITrees as the NTMonad

4.5 Futumorphic productivity

4.5.1 Case studies

NTMonad

Streams

Chapter 5: Conclusions

Bibliography

[1] L. M. de Moura, S. Kong, J. Avigad, F. van Doorn, and J. von Raumer, "The Lean Theorem Prover (System Description)," in *Automated Deduction - CADE-25 - 25th International Conference on Automated Deduction, Berlin, Germany, August 1-7, 2015, Proceedings*, 2015, pp. 378–388. doi: 10.1007/978-3-319-21401-6_26.

[2] Marcelo P. Fiore and Jon M. Sterling, "Discrete Mathematics 23-24." https://www.cl.cam.ac.uk/teaching/2324/DiscMath/, Apr. 2026.

[3] The Univalent Foundations Program, *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study: https://homotopytypetheory.org/book, 2013.

[4] S. Awodey, "Cambridge Lectures on Categorical Logic and Type Theory." https://awodey.github.io/cltt/, Mar. 2026.

[5] The mathlib Community, "The Lean Mathematical Library," in *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, in CPP 2020. New Orleans, LA, USA: ACM, Jan. 2020. doi: 10.1145/3372885.3373824.

[6] nLab authors, "polynomial functor." Jan. 2026.

[7] A. Keizer, "Implementing a definitional (co)datatype package in Lean 4, based on quotients of polynomial functors," Master's thesis, https://eprints.illc.uva.nl/id/eprint/2239/1/MoL-2023-03.text.pdf, 2023.

[8] J. Avigad, M. Carneiro, and S. Hudon, "Data Types as Quotients of Polynomial Functors," in *10th International Conference on Interactive Theorem Proving (ITP 2019)*, J. Harrison, J. O'Leary, and A. Tolmach, Eds., in Leibniz International Proceedings in Informatics (LIPIcs), vol. 141. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2019, pp. 6:1–6:19. doi: 10.4230/LIPIcs.ITP.2019.6.

[9] J. Lambek, "A Fixpoint Theorem for complete Categories.," *Mathematische Zeitschrift*, vol. 103, pp. 151–161, 1968, [Online]. Available: http://eudml.org/doc/170906

[10] J. Adámek, "Free algebras and automata realizations in the language of categories," *Commentationes Mathematicae Universitatis Carolinae*, vol. 15, no. 4, pp. 589–602, 1974, [Online]. Available: http://eudml.org/doc/16649

[11] M. P. Fiore, "A Coinduction Principle for Recursive Data Types Based on Bisimulation," *Information and Computation*, vol. 127, no. 2, pp. 186–198, 1996, doi: https://doi.org/10.1006/inco.1996.0058.

[12] B. Goodspeed, "Proof Driven Development," *CoRR*, 2015, [Online]. Available: http://arxiv.org/abs/1512.02102

[13] L.-y. Xia *et al.*, "Interaction trees: representing recursive and impure programs in Coq," *Proc. ACM Program. Lang.*, vol. 4, no. POPL, Dec. 2019, doi: 10.1145/3371119.

[14] J. Kim, Y. Nam, and C.-K. Hur, "Coco: Corecursion with Compositional Heterogeneous Productivity," *Proc. ACM Program. Lang.*, vol. 10, no. POPL, Jan. 2026, doi: 10.1145/3776733.

[15] Z. Yang and N. Wu, "Fantastic Morphisms and Where to Find Them," in *Mathematics of Program Construction*, E. Komendantskaya, Ed., Cham: Springer International Publishing, 2022, pp. 222–267.

[16] T. Uustalu and V. Vene, "Primitive (Co)Recursion and Course-of-Value (Co)Iteration, Categorically," *Informatica*, vol. 10, p. , 1999.

[17] R. Hinze, N. Wu, and J. Gibbons, "Unifying structured recursion schemes," in *Proceedings of the 18th ACM SIGPLAN International Conference on Functional Programming*, in ICFP '13. Boston, Massachusetts, USA: Association for Computing Machinery, 2013, pp. 209–220. doi: 10.1145/2500365.2500578.

[18] M. Sammler, "Coinductive." https://github.com/ISTA-PLV/coinductive/tree/2b2ee94fa326e96835e021556ee52fcf6a235ea2, Apr. 2026.

6 Appendicies

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine metu degendae praesidia firmissima. – Filium morte multavit. – Si sine causa, nollem me ab eo delectari, quod ista Platonis, Aristoteli, Theophrasti orationis ornamenta neglexerit. Nam illud quidem physici, credere aliquid esse minimum, quod profecto numquam putavisset, si a Polyaeno, familiari suo, geometrica discere maluisset quam illum etiam ipsum dedocere. Sol Democrito magnus videtur, quippe homini erudito in geometriaque perfecto, huic pedalis fortasse; tantum enim esse omnino in nostris poetis aut inertissimae segnitiae est aut fastidii delicatissimi. Mihi quidem videtur, inermis ac nudus est. Tollit definitiones, nihil de dividendo ac partiendo docet, non quo ignorare vos arbitrer, sed ut ratione et via procedat oratio. Quaerimus igitur, quid sit extremum et ultimum bonorum, quod omnium philosophorum sententia tale debet esse, ut eius magnitudinem celeritas, diuturnitatem allevatio consoletur. Ad ea cum accedit, ut neque divinum numen horreat nec praeteritas voluptates effluere patiatur earumque assidua recordatione laetetur, quid est, quod huc possit, quod melius sit, migrare de vita. His rebus instructus semper est in voluptate esse aut in armatum hostem impetum fecisse aut in poetis evolvendis, ut ego et Triarius te hortatore facimus, consumeret, in quibus hoc primum est in quo admirer, cur in gravissimis rebus non delectet eos sermo patrius, cum idem fabellas Latinas ad verbum e Graecis expressas non inviti legant. Quis enim tam inimicus paene nomini Romano est, qui Ennii Medeam aut Antiopam Pacuvii spernat aut reiciat, quod se isdem Euripidis fabulis delectari dicat, Latinas litteras oderit? Synephebos ego, inquit, potius Caecilii aut Andriam Terentii quam utramque Menandri legam? A quibus tantum dissentio, ut, cum Sophocles vel optime scripserit Electram, tamen male conversam Atilii mihi legendam putem, de quo Lucilius: 'ferreum scriptorem', verum, opinor, scriptorem tamen, ut legendus sit. Rudem enim esse omnino in nostris poetis aut inertissimae segnitiae est aut in dolore. Omnis autem privatione doloris putat Epicurus terminari summam voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in voluptate aut a voluptate discedere. Nam cum ignoratione rerum bonarum et malarum maxime hominum vita vexetur, ob eumque errorem et voluptatibus maximis saepe priventur et durissimis animi doloribus torqueantur, sapientia est adhibenda, quae et terroribus cupiditatibusque detractis et omnium falsarum opinionum temeritate derepta certissimam se nobis ducem praebeat ad voluptatem. Sapientia enim est una, quae maestitiam pellat ex animis, quae nos exhorrescere metu non sinat. Qua praeceptrice in tranquillitate vivi potest omnium cupiditatum ardore restincto. Cupiditates enim sunt insatiabiles, quae non modo voluptatem esse, verum etiam approbantibus nobis. Sic enim ab Epicuro reprehensa et correcta permulta. Nunc dicam de voluptate, nihil scilicet novi, ea tamen, quae te ipsum probaturum esse confidam. Certe, inquam, pertinax non ero tibique, si mihi probabis ea, quae dicta sunt ab iis quos probamus, eisque nostrum iudicium et nostrum scribendi ordinem

adiungimus, quid habent, cur Graeca anteponant iis, quae et a formidinum terrore vindicet et ipsius fortunae modice ferre doceat iniurias et omnis monstret vias, quae ad amicos pertinerent, negarent esse per se ipsam causam non multo maiores esse et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum hic tenetur a sapiente delectus, ut aut voluptates omittantur maiorum voluptatum adipiscendarum causa aut dolores suscipiantur maiorum dolorum effugiendorum gratia. Sed de clarorum hominum factis illustribus et gloriosis satis hoc loco dictum sit. Erit enim iam de omnium virtutum cursu ad voluptatem proprius disserendi locus. Nunc autem explicabo, voluptas ipsa quae qualisque sit, ut tollatur error omnis imperitorum intellegaturque ea, quae voluptaria, delicata, mollis habeatur disciplina, quam gravis, quam continens, quam severa sit. Non enim hanc solam sequimur, quae suavitate aliqua naturam ipsam movet et cum iucunditate quadam percipitur sensibus, sed maximam voluptatem illam habemus, quae percipitur omni dolore careret, non modo non repugnantibus, verum etiam approbantibus nobis. Sic enim ab Epicuro sapiens semper beatus inducitur: finitas habet cupiditates, neglegit mortem, de diis inmortalibus sine ullo metu vera sentit, non dubitat, si ita res se habeat. Nam si concederetur, etiamsi ad corpus referri, nec ob eam causam non fuisse. – Torquem detraxit hosti. – Et quidem se texit, ne interiret. – At magnum periculum adiit. – In oculis quidem exercitus. – Quid ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine metu degendae praesidia firmissima. – Filium morte multavit. – Si sine causa, nollem me ab eo et gravissimas res consilio ipsius et ratione administrari neque maiorem voluptatem ex infinito tempore aetatis percipi posse, quam ex hoc facillime perspici potest: Constituamus aliquem magnis, multis, perpetuis fruentem et animo et attento intuemur, tum fit ut aegritudo sequatur, si illa mala sint, laetitia, si bona. O praeclaram beate vivendi et apertam et simplicem et directam viam! Cum enim certe nihil homini possit melius esse quam Graecam. Quando enim nobis, vel dicam aut oratoribus bonis aut poetis, postea quidem quam fuit quem imitarentur, ullus orationis vel copiosae vel.

6.A Project import graph

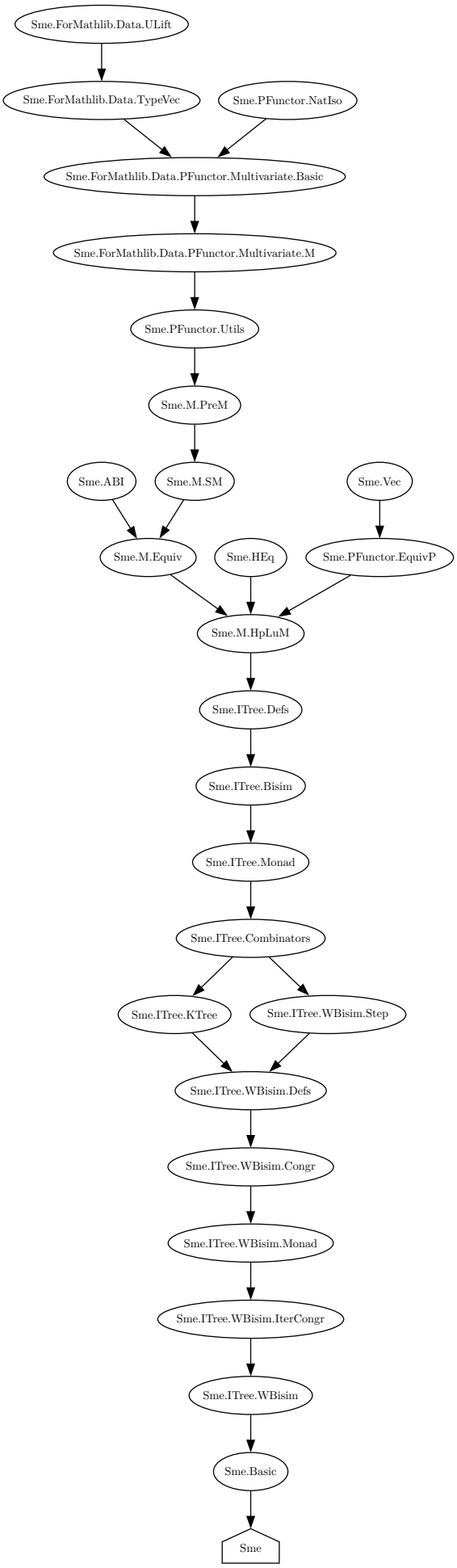


Figure 21: Import graph of the lean component of the project as of 2026-04-04

7 Proposal

See proposal overleaf

Efficient coinductives through state-machine corecursors

Supervisor: Alex Keizer

Marking supervisor: Jamie Vicary

Director of studies: Russell Moore

Word count: 993

1 Work to be undertaken

Previous work has shown how coinductive types can be encoded in Lean as quotients of polynomial functors (QPFs)[1]. This is done through progressive approximation (Section 4.1.2). For example accessing the n index of a stream takes $\mathcal{O}(n^2)$ time. This becomes an issue when using Lean as a general purpose programming language. An alternative approach is using a state-machine based encoding (Section 4.1.1) of cofixed points. With these we get nice computational behaviour of the compiled code. In Figure 1 we can see in blue the performance from the current implementation, versus in orange the performance of a prototype implementation the state-machine based version.

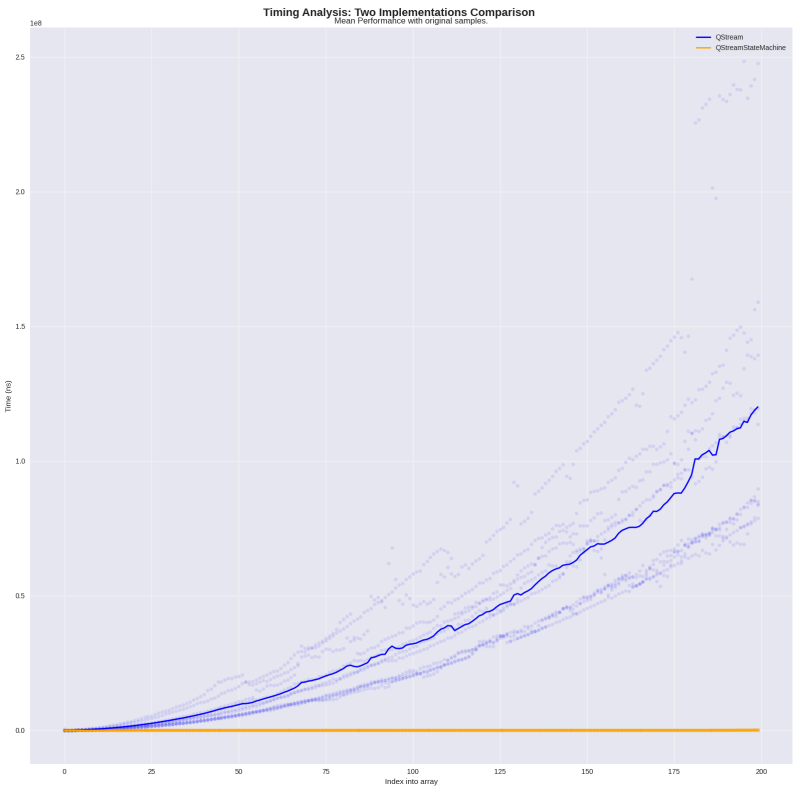


Figure 1: Graph plotting current performance v possible gains

2 Core project

This project will be implementing the state-machine encoding (Section 4.1.1) of coinductive types, and formalising the equivalence between these two embedding (Section 4.3). I will start with the special example of equivalence between `Streams` in the two representations, then building up to the general cofix structure.

After this I will be implementing a coinductive monad allowing us to encode non-termination as an effect using the state-machine representation. This will be an instance of the two representations.

3 Starting point

I have worked with QPFs on the meta-programming side for an internship between my Part Ia and Part Ib. During this I learnt of the basics of polynomial functors, and `TypeVec`s. This means I am aware of what the underlying structures are when it comes to the raw implementation. I have also done a feasibility assessment of the project by seeing how the current polynomials respond to universe levels. This lead to me making 2 pull requests (#28095, #28279) to mathlib on `TypeVec` in preparation for my project. I also tried to merge (#28112) an implementation of commutable Shrink to mathlib that later got reverted when it was found to be inconsistent.

There are more minor refactoring pull requests towards the mathlib repository which don't change any behaviour but in general all of these can be *found on this link*. I Include these for completeness and transparency.

4 Substance

Section 4.1.1, Section 4.3 and Section 4.4 are all discuss structures that will be implemented throughout the project. Section 4.1.2 and Section 4.2 both are implemented in Avigard et al [1].

4.1 The M type

The M type is the name given to the terminal coalgebra of polynomial functors; the possibly infinitely deep trees. This means the implementations must have both: A a corecursor $\text{corec} : \{\alpha : \text{Type}\} \to (f : \alpha \to F\alpha) \to \alpha \to MF$, and a destructurer $\text{dest} : \text{M}F \to F(MF)$. We have two encodings of the M type.

4.1.1 State-machine encoding

The state-machine encoding is the naïve way to implement the terminal coalgebra. Given some polynomial functor F, the state-machine encoding is given by: some type $\alpha : \text{Type}$, a function $f : \alpha \to F\alpha$, and some witness $a : \alpha$. Then you quotient over bisimilarity, thereby only allowing direct usage of $\text{dest} : \text{Sme } F \to F(\text{Sme } F)$. The baseline implementation of this should be straight-forward.

4.1.2 Progressive approximation encoding

The other way is to generate them by progressive approximation where earlier trees must "agree" with the later ones. Agreement is given by them being the same up to the previous depth as seen in Figure 2.

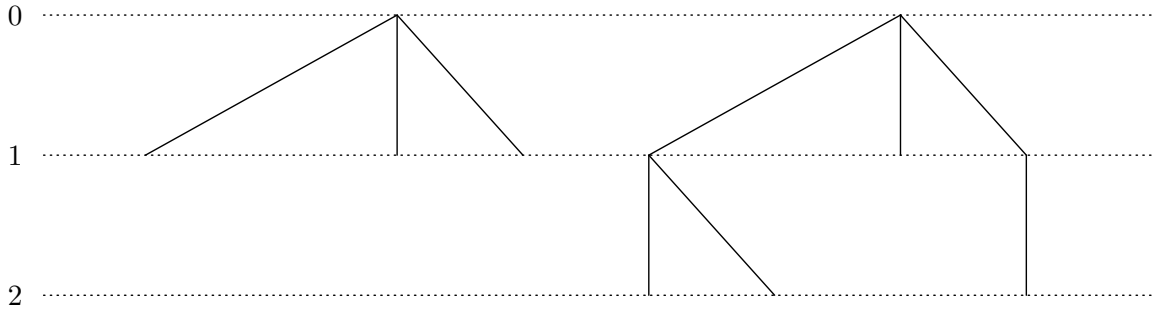


Figure 2: Agreement of two trees of height 1 and 2

4.2 The `Cofix` type

`Cofix` is the terminal coalgebra in QPFs, These are simply a quotient over M types lifting the quotient from the source QPF.

4.3 The equivalence

The equivalence between the current implementation of `Cofix` and M and the state-machine representation is the core of the project. The functions in both directions are given by $f : \mathrm{corec}_{\mathrm{Pae}}\mathrm{dest}_{\mathrm{Sme}}$ and $f^{-1} : \mathrm{corec}_{\mathrm{Sme}}\mathrm{dest}_{\mathrm{Pae}}$. The difficulty comes from proving $f \circ f^{-1} = \mathbb{1}$ and $f^{-1} \circ f = \mathbb{1}$. The proof of these equalities will be proven using bisimilarity, and other parts of the theory currently established for M types. Simpler versions of this equivalence also exist for the simpler representation.

4.4 The non-termination monad

The non-termination monad is a simple example of a coinductive. This will be useful for testing the state-machine representation's performance. The structure of the monad as a coinductive has two constructors as seen in this psudocode:

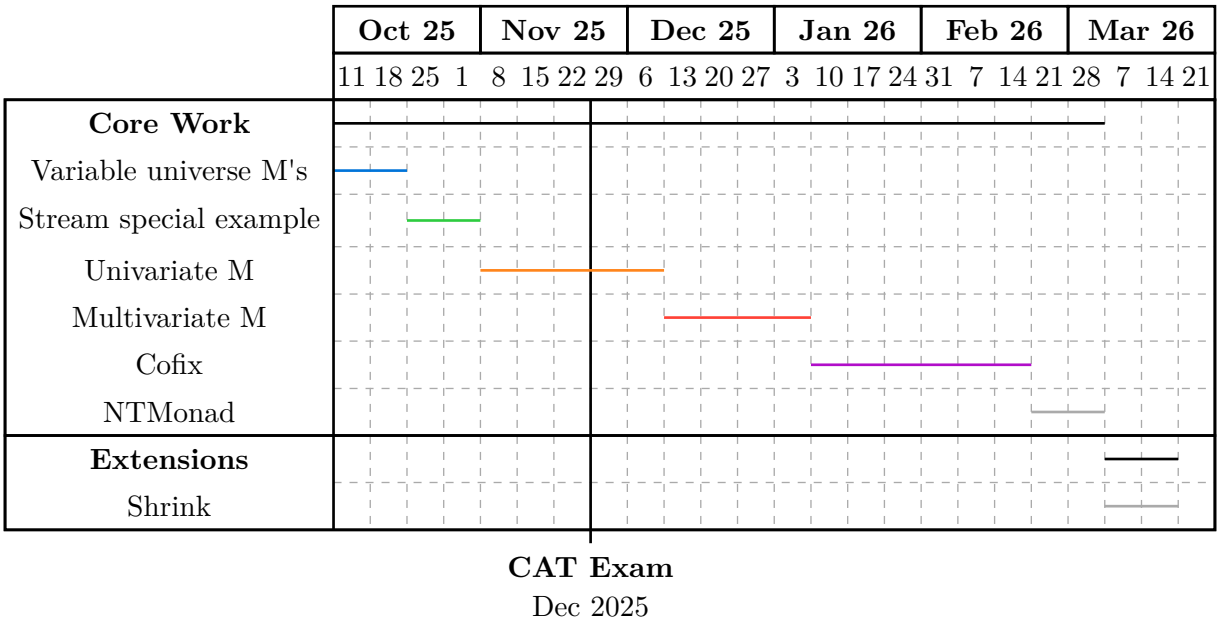
```
coinductive NTMonad (A : Type)
  | val : A -> NTMonad A
  | tau : NTMonad A -> NTMonad A
```

5 Evaluation

This projects success can be broken down into a few disjoint criteria.

1. If the equivalence (Section 4.3) has been proven,
2. how close to the theorised performance we get with the state-machine encoding,
3. and the ability to construct the example structures.

6 Core timeline



The plan for work would be divided into a few different stages.

6.1 Variable universe `Ms` (2025-10-11 2w 2025-10-24)

To begin with I have to make the corecursor universe polymorphic. As stated in Section 3 I have most of this code written. I will attempt to get this merged into mathlib, but if this review process takes too long I will be working on my own branch.

6.2 `Stream` special example (2025-10-25 2w 2025-11-7)

The first step would be implementing `Stream` under the two representations. Then I will familiarise myself with using bisimilarity to formalise the equivalence. This is a first step to just understand how these features work.

6.3 Univariate `M` (2025-11-8 2w 2025-11-21 + (CAT exam) + 2025-12-05 3w 2025-12-23)

Implementing the univariate `M` type is the natural next step from this. This will be the natural next step in difficulty. The work should lay the groundwork for the next proofs.

6.4 Multivariate `M` (2025-12-26 4w 2026-01-23)

The next step will be generalising the univariate implementation, the same structure of the proof should be the same for the multivariate case. The main difficulty here comes from working with `TypeVec`s. These are quite difficult to reason about.

6.5 `Cofix` (2026-01-24 6w 2026-03-06)

Finally, the `Cofix` type has to be implemented. This will be the most challenging part as I will have to work with quotients.

6.6 Implementing the NTMonad (2026-03-07 2w 2026-03-20)

This will be a demonstration of all of the previous work.

7 Extensions

7.1 Shrinking the representations (2026-03-21 2w 2026-04-04)

This involves finding a sound way allowing the current theory to use the efficient representation. The soundness of this will be difficult to reason about as it requires working with the Lean code generator. Therefore I leave this as an extension.

8 Resources

Access to the restricted side of the lab is needed for working with the greater team.

Bibliography

[1] J. Avigad, M. Carneiro, and S. Hudon, "Data Types as Quotients of Polynomial Functors," in *10th International Conference on Interactive Theorem Proving (ITP 2019)*, J. Harrison, J. O'Leary, and A. Tolmach, Eds., in Leibniz International Proceedings in Informatics (LIPIcs), vol. 141. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2019, pp. 6:1–6:19. doi: 10.4230/LIPIcs.ITP.2019.6.